Fakultas Rekayasa Industri

ENTERPRISE APPLICATION DEVELOPMENT



Proyek Perangkat Lunak

Modul 2

Version Control System

EAD Laboratory

Materi Modul 2 - Version Control System

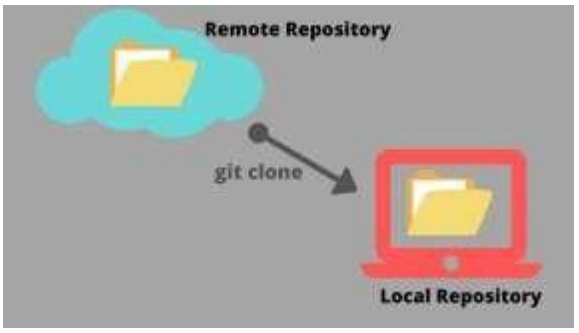
Github

Github merupakan platform *Version Control System* (VCS) dan kolaborasi dalam mendevelop perangkat lunak dengan basis web. Github memungkinkan pengembang berkolaborasi secara lebih efektif dengan menyediakan alat untuk mengelola perubahan yang mungkin saling bertentangan dari beberapa pengembang. Github memungkinkan pengembang untuk mengubah, mengadaptasi, dan meningkatkan perangkat lunak dari repositori.

a. Commit

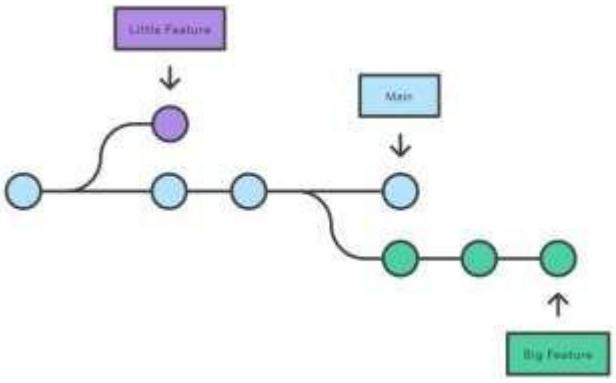
Commit adalah proses untuk menyimpan setiap perubahan yang telah kita lakukan pada file ke dalam *database* lokal Git. Bayangkan *commit* sebagai sebuah "rekaman" atau *snapshot* dari progres kerja teman-teman. Dengan melakukan *commit*, kita memiliki jejak sejarah (history) yang memungkinkan kita untuk kembali ke versi sebelumnya jika terjadi kesalahan di kemudian hari.

b. Clone



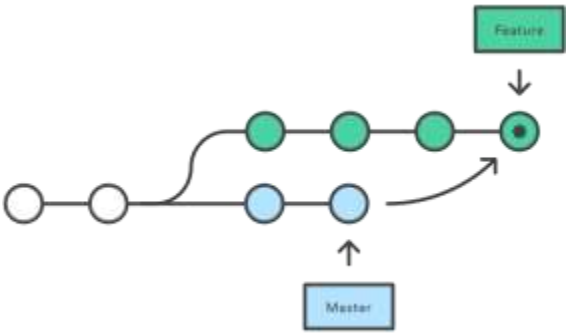
Clone adalah proses menyalin repositori yang ada di server (*remote*) ke komputer lokal kita. Berbeda dengan sekadar mengunduh (*download*) file, *clone* akan mengambil seluruh sejarah proyek, semua *branch*, dan semua *commit* yang pernah ada.

c. Branch



Branch digunakan untuk mencegah konflik saat bekerja dalam tim. Bayangkan teman-teman sedang bekerja pada suatu *repository* Git yang sama. Jika semua orang langsung mengubah kode utama secara bersamaan, kemungkinan besar akan terjadi bentrokan karena kode yang ditulis satu sama lain berbeda.
Contoh: Si A menulis kode untuk fitur X dengan algoritma tertentu, sedangkan Si B menulis dengan algoritma yang berbeda. Jika mereka langsung melakukan *commit* di tempat yang sama, kode sumber akan menjadi berantakan. Agar tidak terjadi hal seperti ini, teman-teman harus membuat cabang (*branch*) tersendiri. Si A bebas melakukan apapun di *branch*-nya tanpa mengganggu cabang utama (*main/master*).

d. Merge



Merging adalah proses penggabungan sebuah *branch* dengan *branch* yang lain. Biasanya dilakukan ketika sebuah fitur di *branch* tertentu sudah selesai diuji dan siap disatukan ke dalam kode utama.

e. Conflict

Conflict terjadi jika sebuah file dalam dua buah *branch* yang akan digabungkan memiliki perbedaan tepat pada baris yang sama. Dalam kondisi ini, Git tidak bisa menentukan secara otomatis kode mana yang benar, sehingga kita harus memperbaikinya secara manual.

f. Pull Request

Pull Request (PR) adalah istilah yang bisa kita artikan sebagai permintaan untuk menggabungkan kode. Setelah membuat *branch*, teman-teman dapat pindah (*checkout*) ke *branch* yang baru dibuat, lalu melakukan kontribusi seperti *commit* & *push* file yang dikerjakan ke *branch* masing-masing. Jika kontribusi sudah selesai dan ingin digabungkan dengan *branch* utama (*main/master*), maka teman-teman dapat membuat *Pull Request* agar kode tersebut dapat ditinjau (*review*) sebelum benar-benar digabungkan.

Materi Praktik

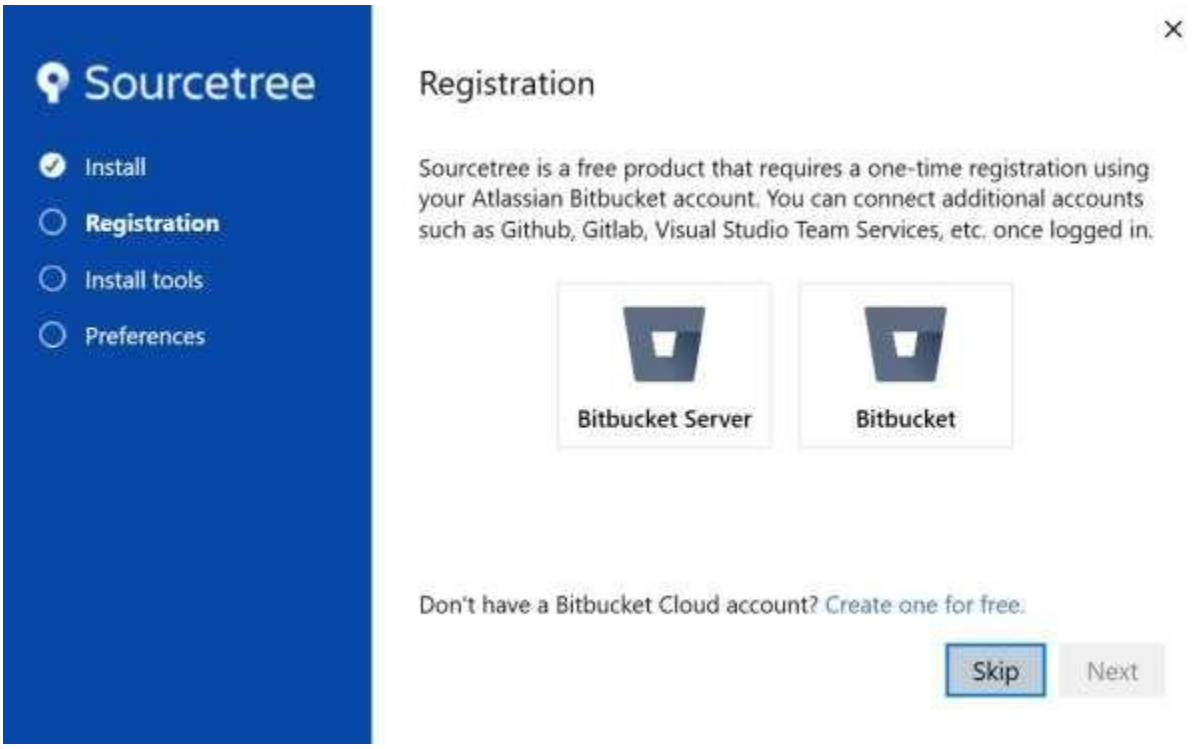
1. Instalasi SourceTree & menghubungkan akun Github

a. Prasyarat

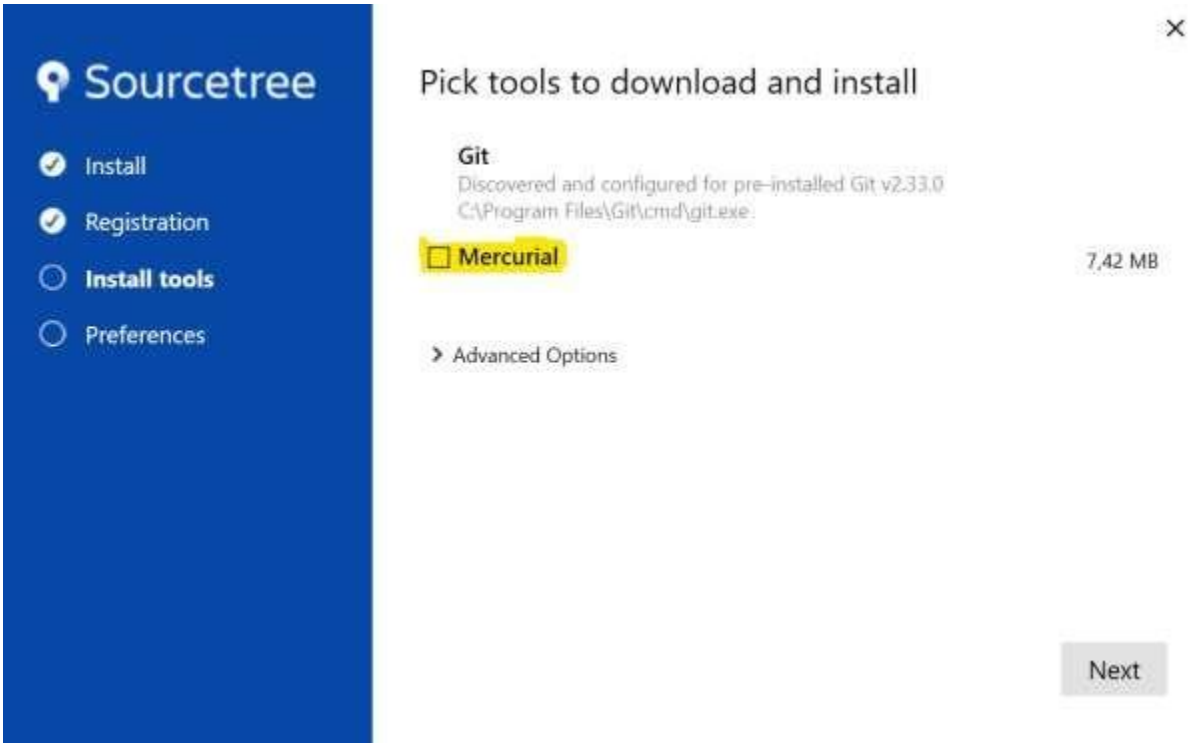
1. Memiliki akun GitHub.
2. Download SourceTree melalui link berikut (sesuaikan dengan OS kalian masing-masing) https://www.sourcetreeapp.com/download-archives
3. Melakukan instalasi SourceTree.

b. Instalasi SourceTree

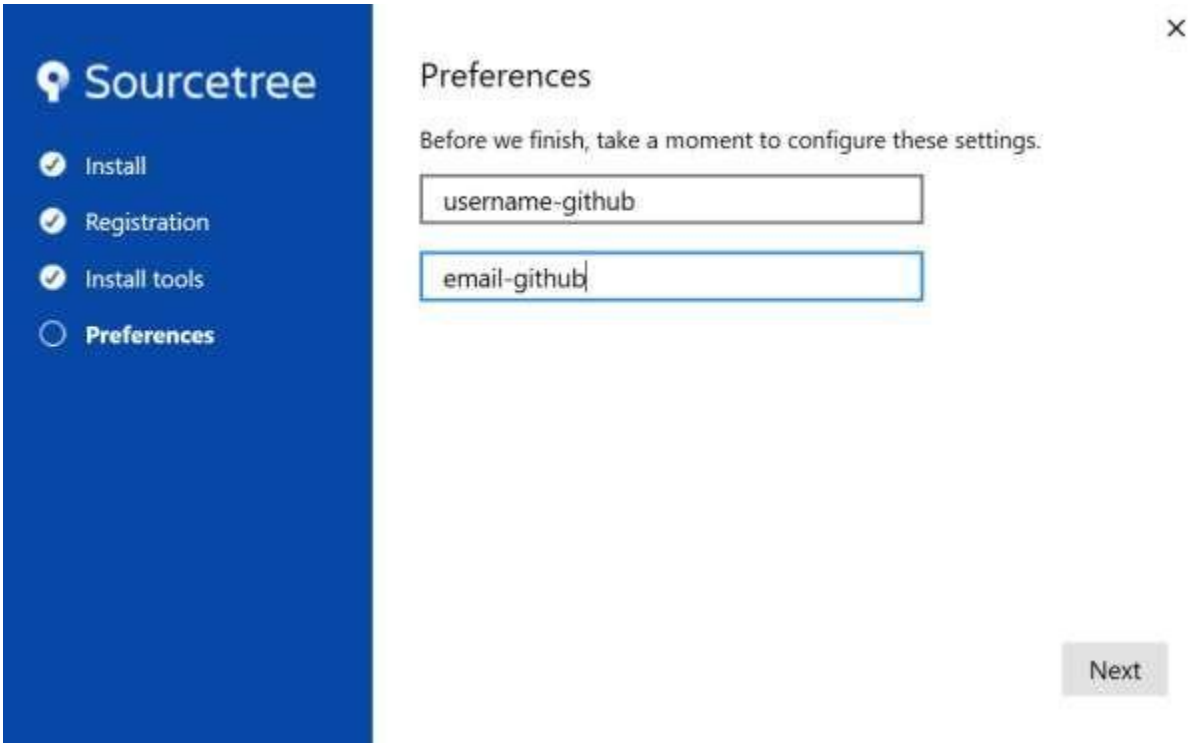
1. Jalankan file SourceTree yang sudah di-download pada link di atas
2. Pada tab Registration pilih 'Skip'



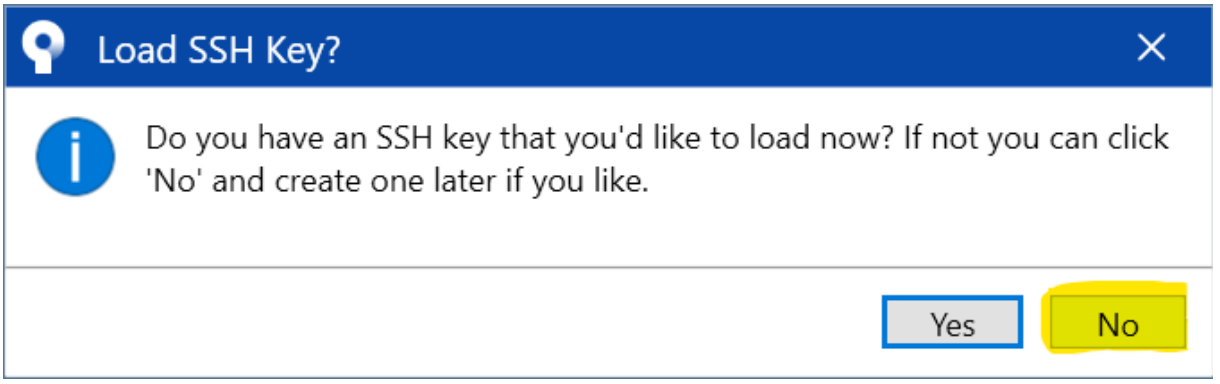
3. Pada tab Install tools **uncheck mercurial** lalu pilih 'Next'



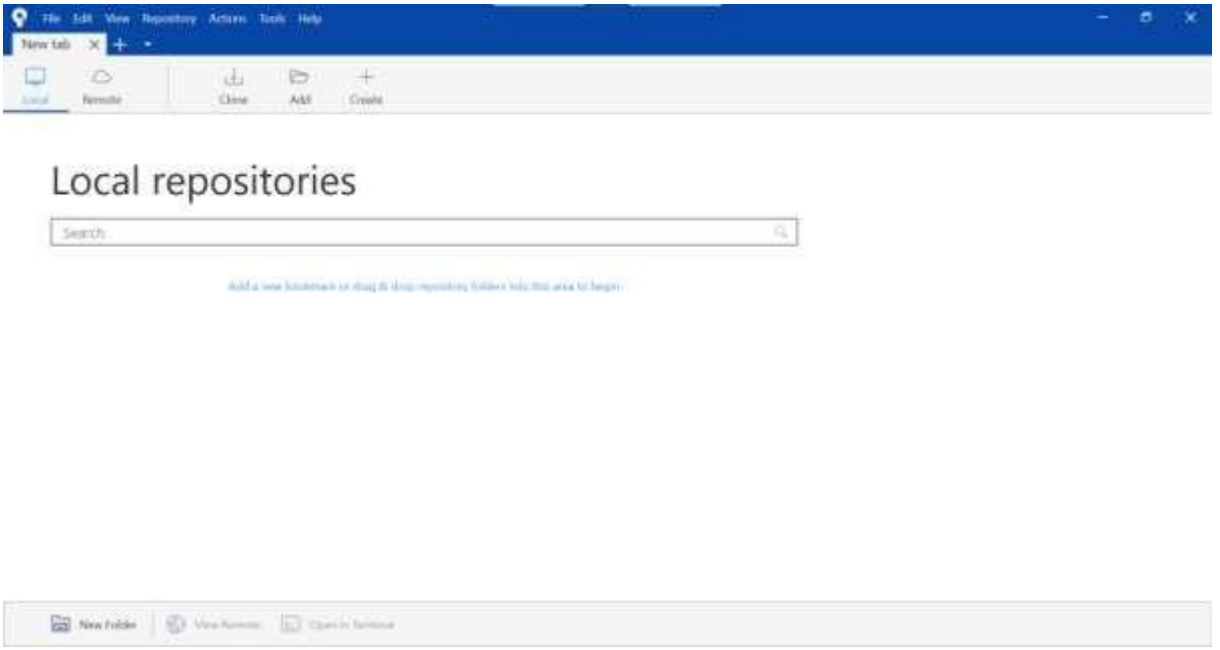
4. Masukkan username dan email Github, lalu pilih next



5. Selanjutnya pilih **No**

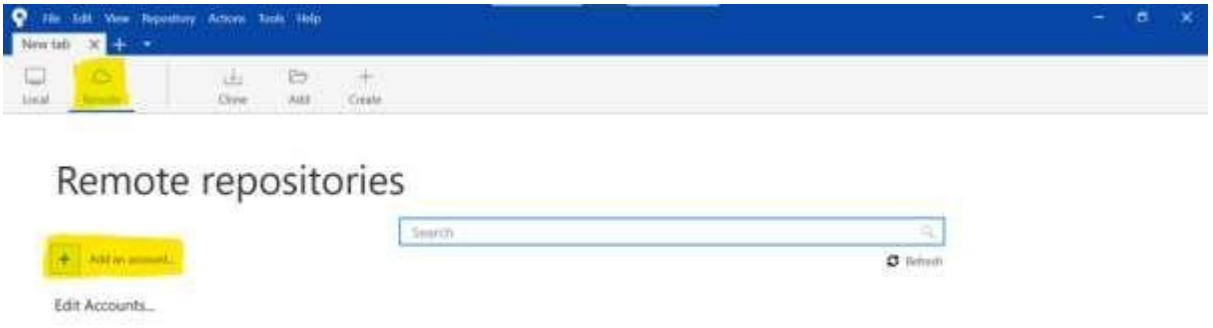


6. Berikut adalah tampilan utama SourceTree

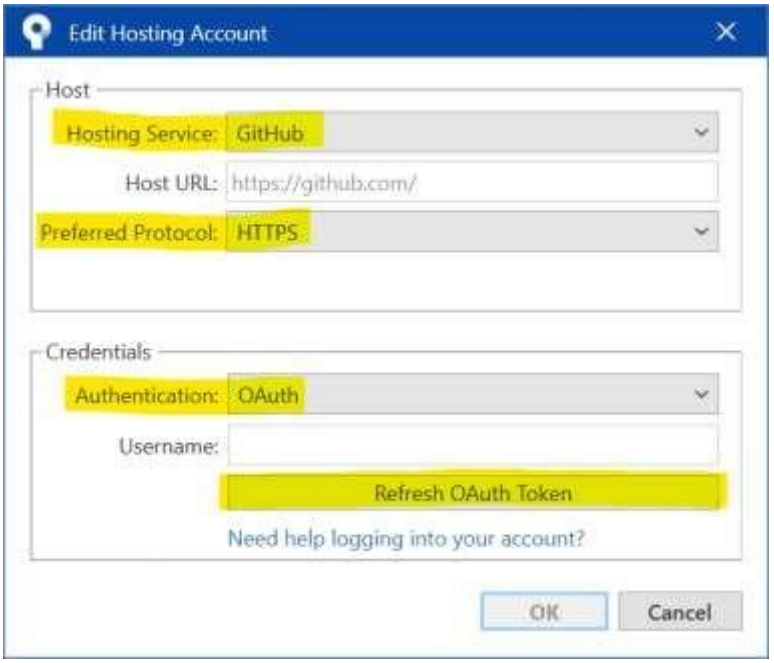


c. Menghubungkan akun Github

1. Masuk ke dalam akun Github dengan cara buka tab Remote pada SourceTree, lalu pilih ‘Add an account’.



2. Kemudian kita dapat menghubungkan akun Github yang kita miliki seperti pada gambar di bawah ini.

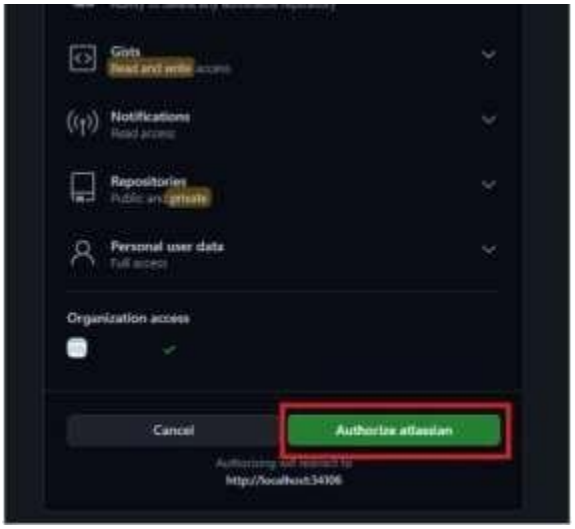


Dalam menghubungkan akun GitHub, ada beberapa hal yang perlu diperhatikan, diantaranya sebagai berikut:

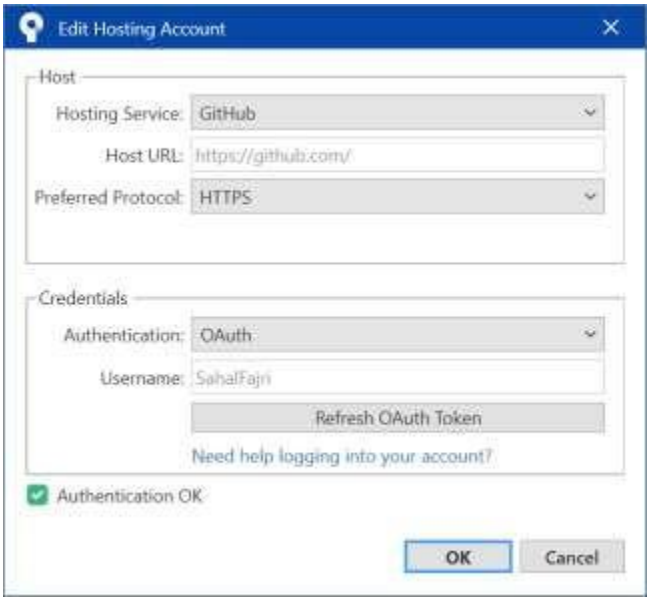
- Hosting Service : GitHub
- Preferred Protocol : HTTPS
- Authentication : OAuth

Setelah dipastikan pengaturannya sesuai, klik tombol ***Refresh OAuth Token*** yang akan mengarahkan pada tampilan konfirmasi authorize (jika sudah masuk akun GitHub pada website), namun jika belum masuk akun GitHub pada website akan diarahkan ke halaman Sign in GitHub untuk masuk akun GitHub terlebih dahulu.

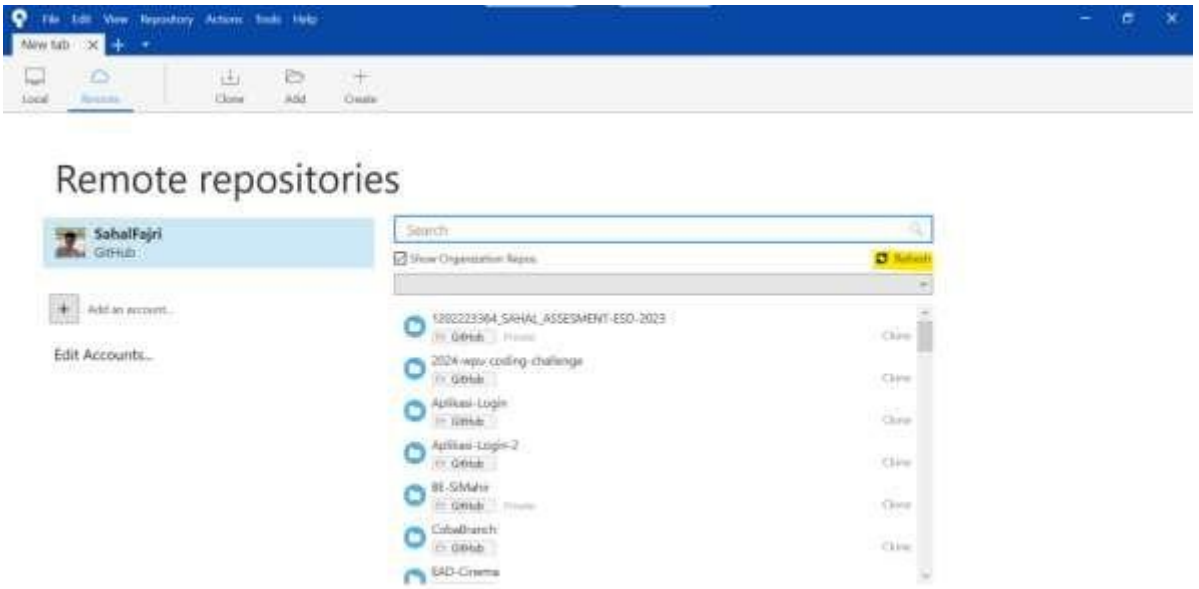
3. Berikut tampilan untuk mengonfirmasi authorize (perizinan) untuk menghubungkan SourceTree dengan GitHub, klik ‘Authorize atlassian’.



Jika sudah berhasil terhubung, pada SourceTree akan tercentang Authentication-nya, lalu klik ‘Ok’.



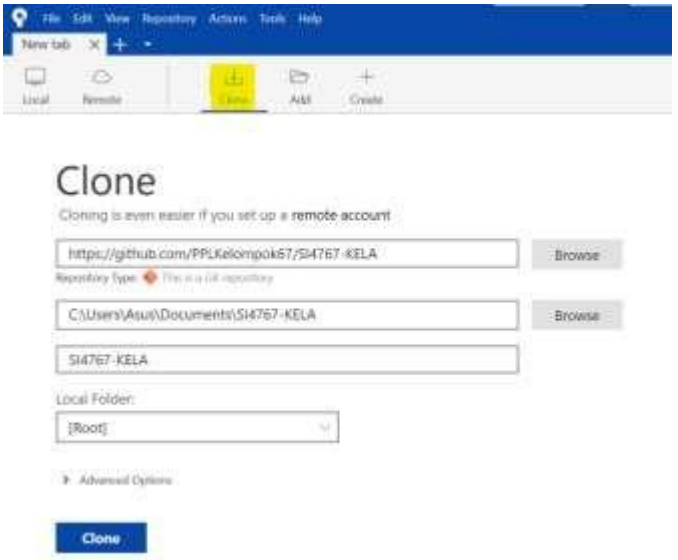
4. Setelah akun GitHub berhasil terhubung, maka kita dapat melihat seluruh *repository* yang telah kita buat sebelumnya, untuk memperbarui daftar *repository* dapat digunakan tombol **Refresh**.



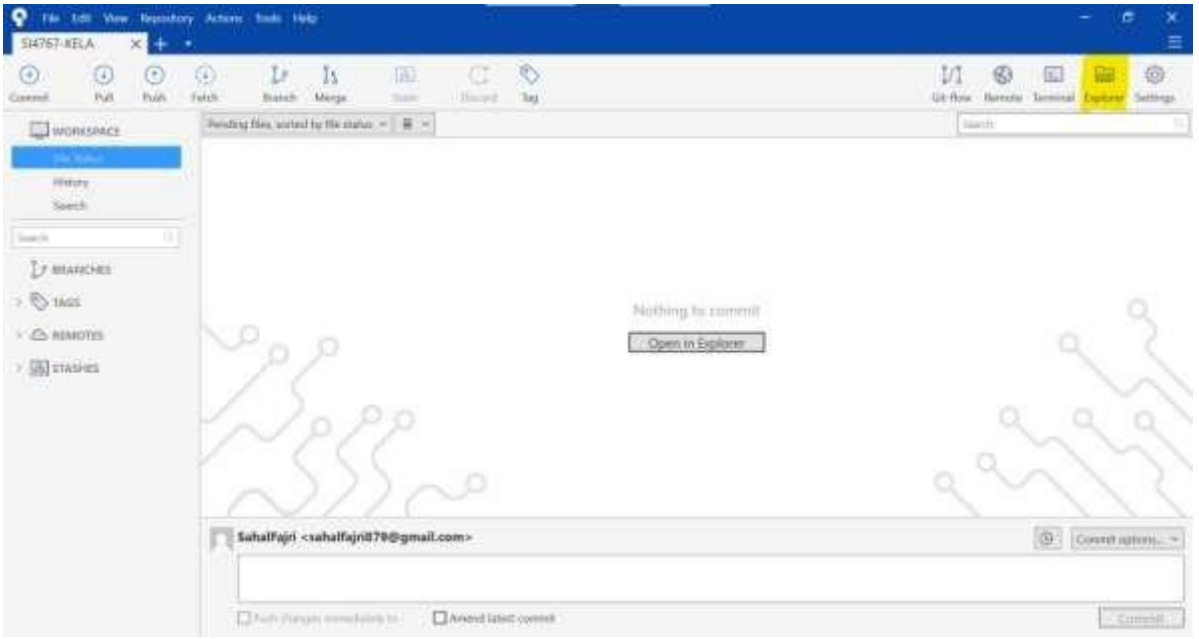
2. Commit, Branch, Merge, Pull Request tanpa Issue Key

a. Commit & Push

1. Sebelum melakukan commit & push, kita melakukan *clone repository* terlebih dahulu **(Pastikan sudah membuat *repository* terlebih dahulu pada Github)**. Dalam melakukan *clone repository*, pastikan *source path remote repository, destination path*, dan nama *local repository* sudah sesuai dengan kebutuhan kita. Selanjutnya jika seluruh pengaturan telah disesuaikan, klik tombol **Clone** dan tunggu hingga proses selesai.



2. Setelah proses clone selesai, maka local *repository* berhasil dibuat. Berikut adalah tampilan dari local *repository* yang kita miliki. Untuk melihat isi *repository* tersebut, klik ‘Explorer’ agar diarahkan pada folder *repository* lokal.



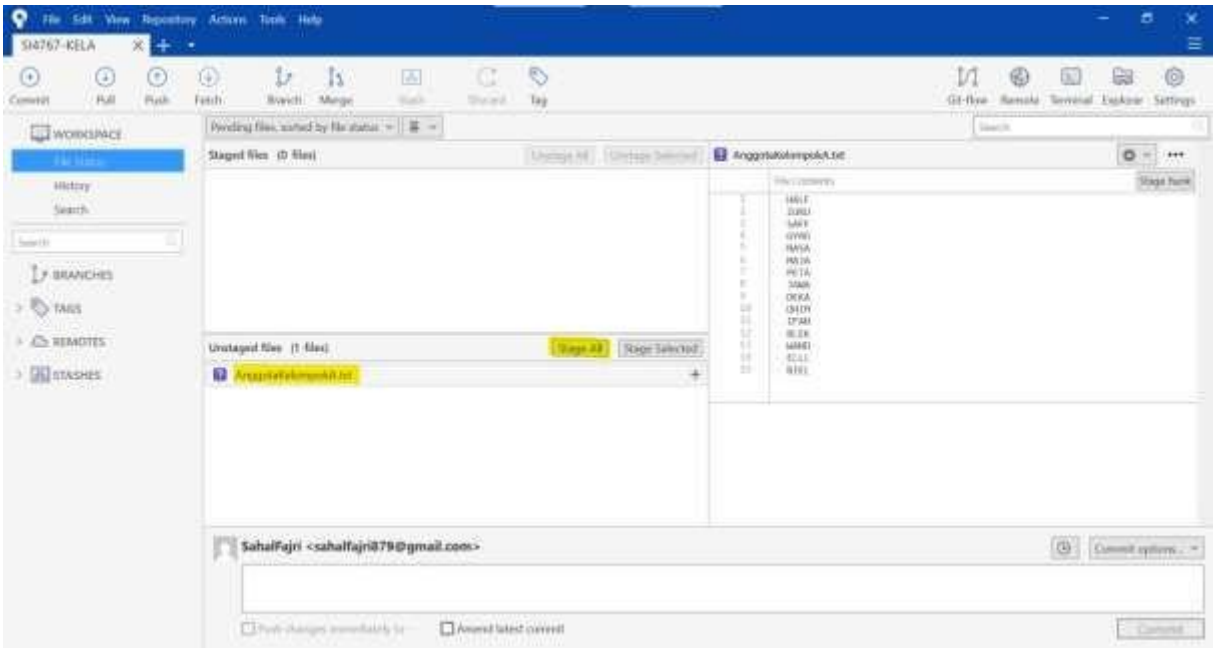
Berikut adalah tampilan folder lokal hasil clone repository. Pastikan folder path mencantumkan nama *repository* yang sesuai dengan yang ada di GitHub.



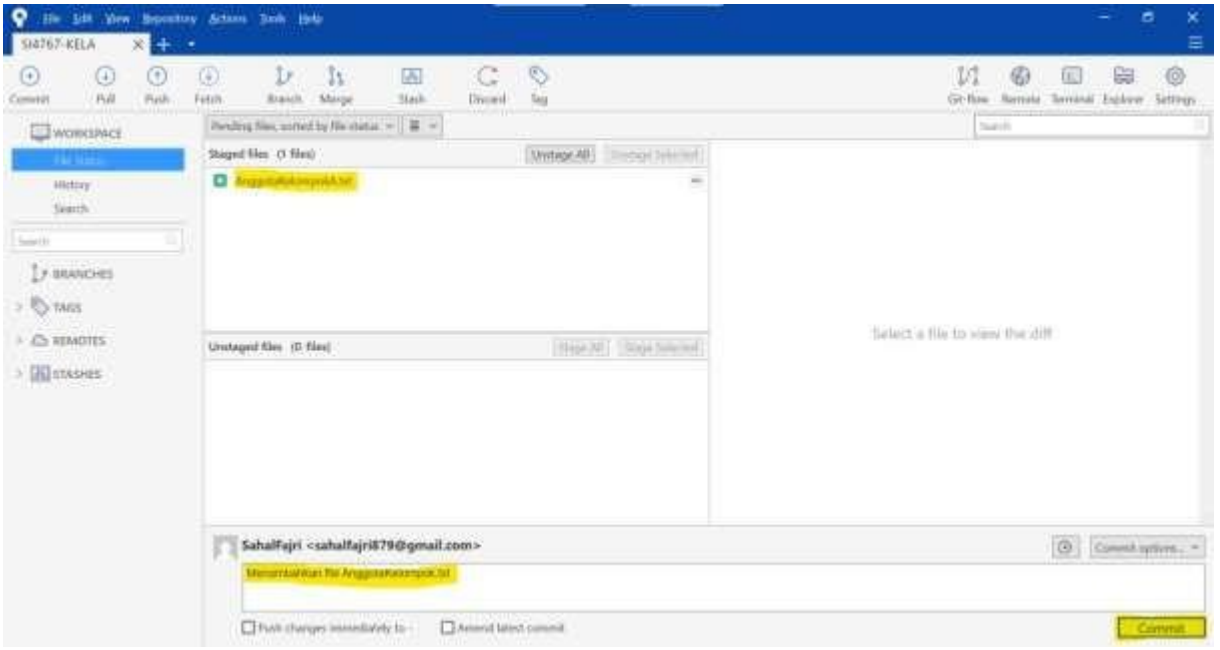
3. Untuk melakukan commit dan push, tambahkan file proyek ke dalam *repository* lokal.



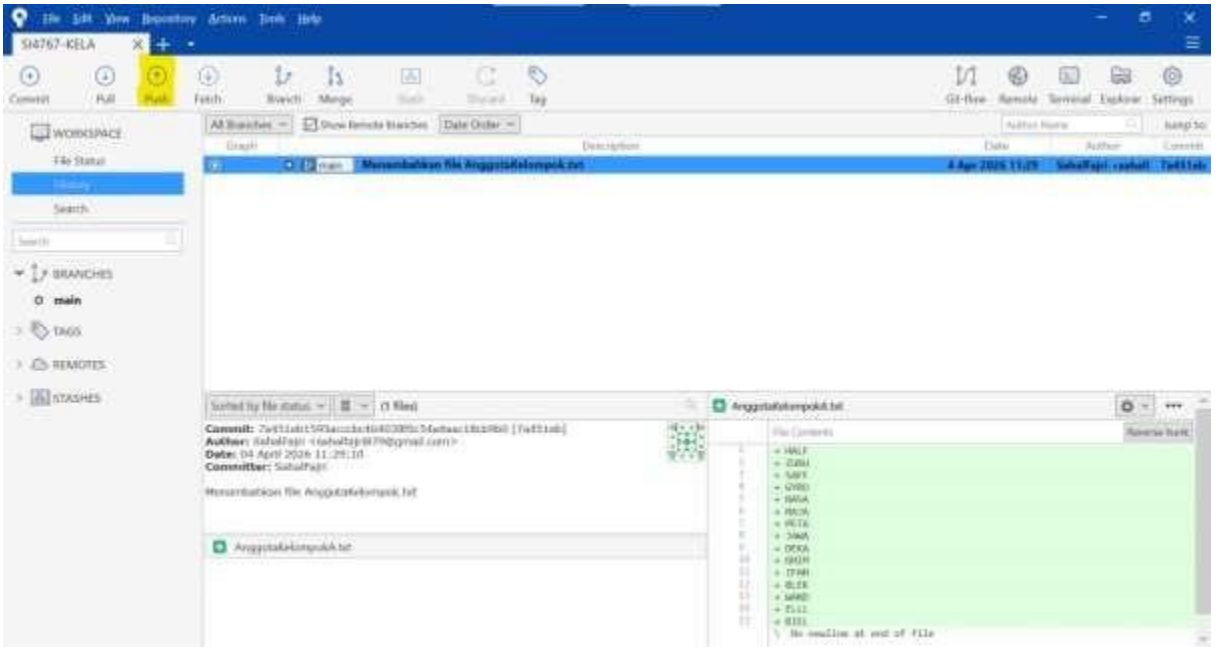
4. Setelah menambahkan file ke dalam *repository* lokal, file tersebut akan muncul di bagian Unstaged Files di SourceTree. Setiap perubahan yang terjadi dalam proyek, baik penambahan, pengeditan, maupun penghapusan file, akan terdeteksi dan ditampilkan di SourceTree. Untuk melakukan commit, kita bisa memilih "Stage All" jika ingin menyertakan semua perubahan, atau "Stage Selected" jika hanya ingin commit beberapa file tertentu.



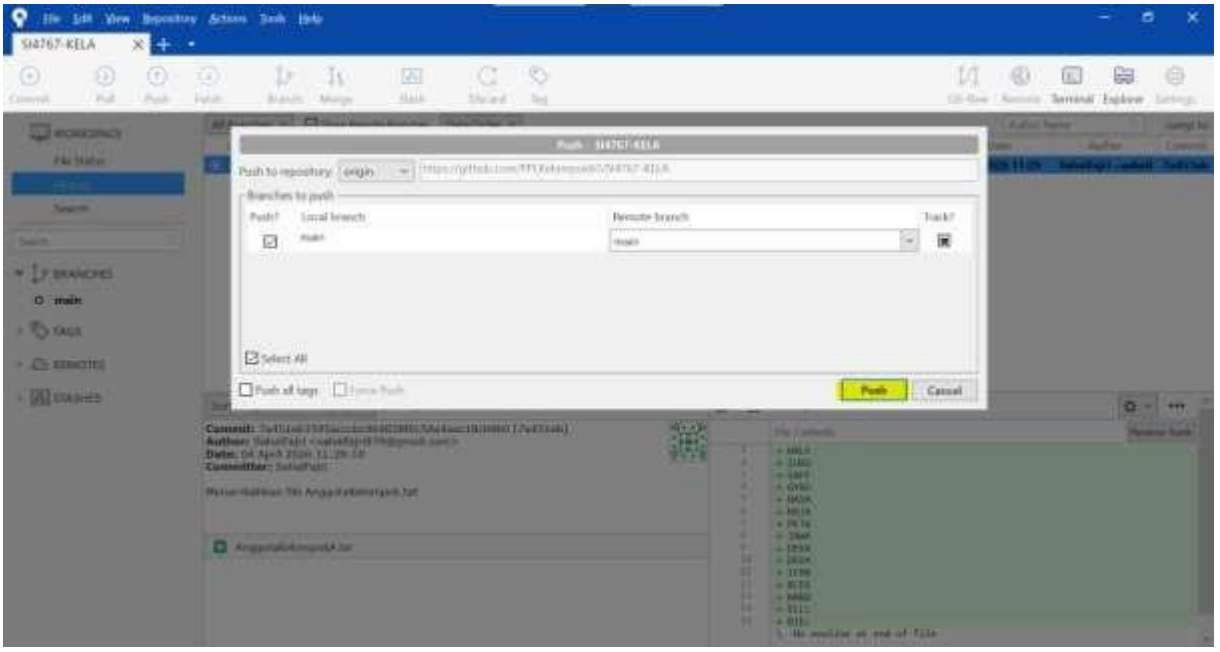
5. Sebelum melakukan commit, pastikan file yang telah dibuat sudah masuk ke dalam Staged Files. Setiap commit sebaiknya disertai dengan pesan yang sesuai dengan perubahan yang dilakukan. Setelah itu, klik "Commit" untuk menyimpan perubahan.



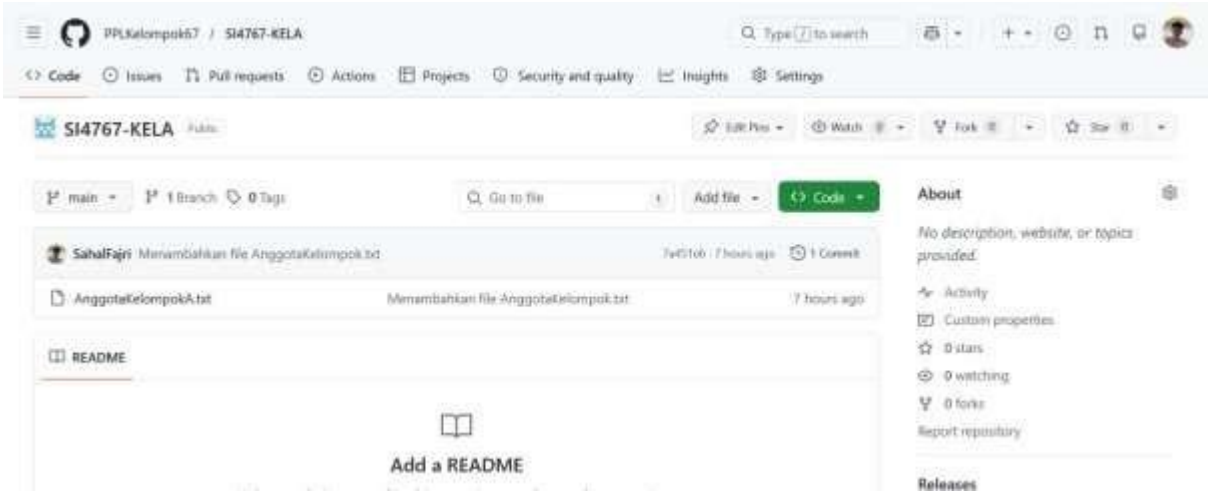
6. Setelah melakukan commit, langkah selanjutnya adalah melakukan push dengan mengklik "Push" di SourceTree. Proses push berfungsi untuk mengunggah perubahan yang telah di-commit ke *repository* GitHub.



7. SourceTree akan menampilkan daftar remote branch yang tersedia. Karena pada *repository* ini belum dibuat branch tambahan, pilih "master/main" sebagai remote branch. Selanjutnya, klik "Push" untuk memulai proses unggah file ke *repository* GitHub dan tunggu hingga proses selesai. **(JIKA TERKENDALA ERROR 403 BISA MENGIKUTI PADA BAGIAN RESOLVE ERROR)**

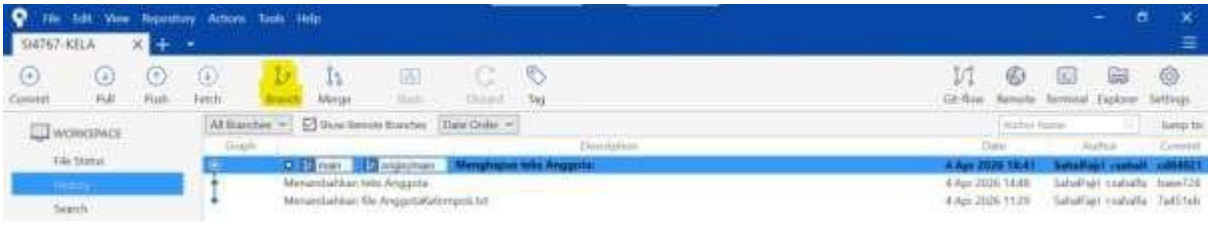


8. Jika sudah berhasil push, maka di *repository* GitHub sudah terlihat file yang kita tambahkan.

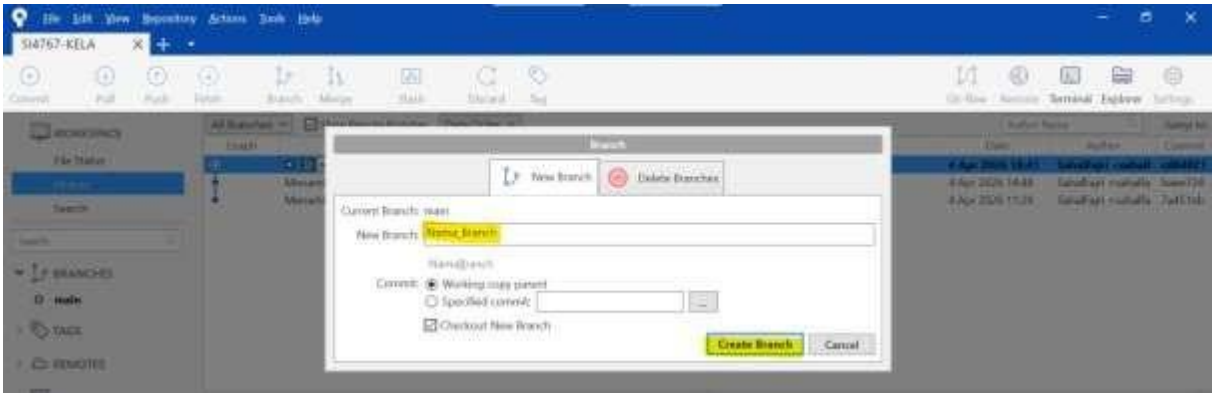


b. Membuat Branch, Commit, dan Push dari Branch

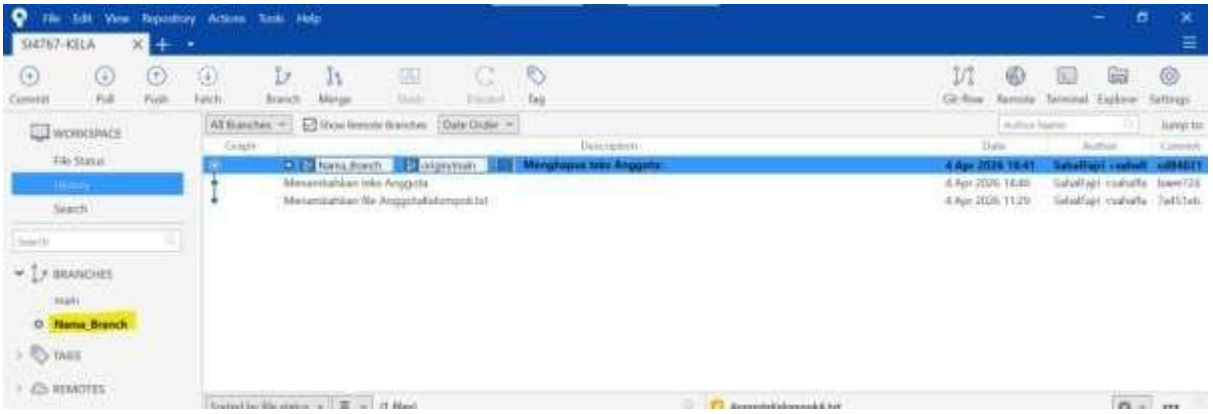
1. Untuk membuat branch baru, klik “Branch” pada menu atas.



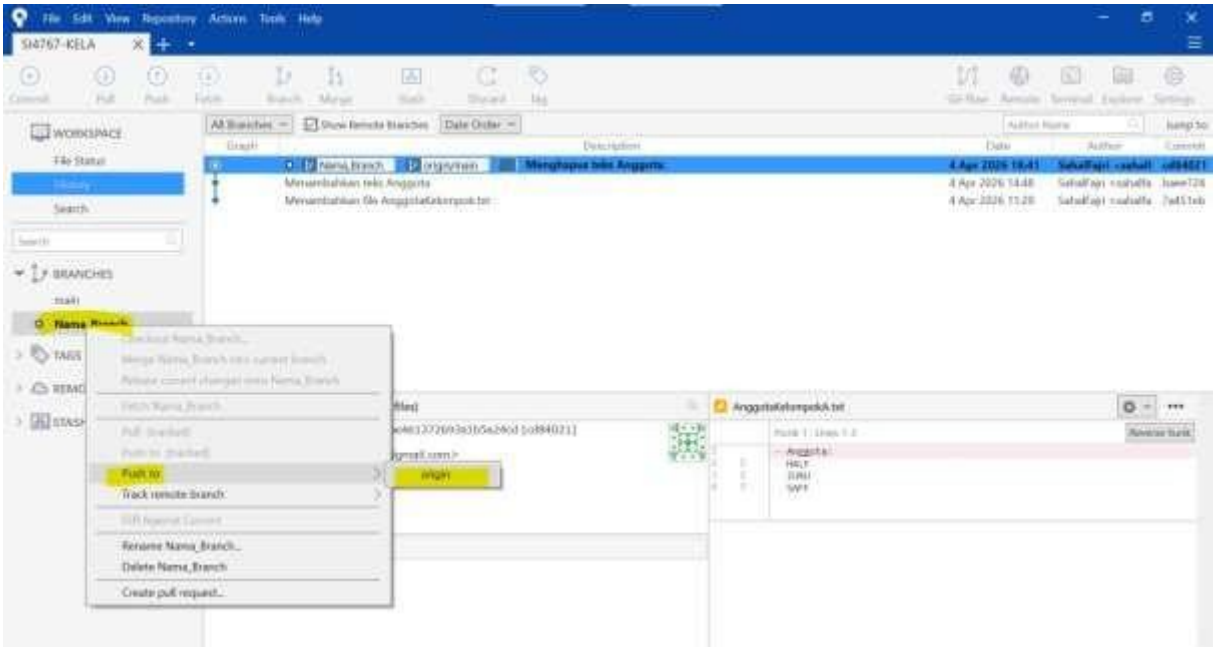
2. Kemudian akan muncul dialog box seperti dibawah ini. Kemudian inputkan nama branch yang ingin dibuat, lalu klik “Create Branch”.



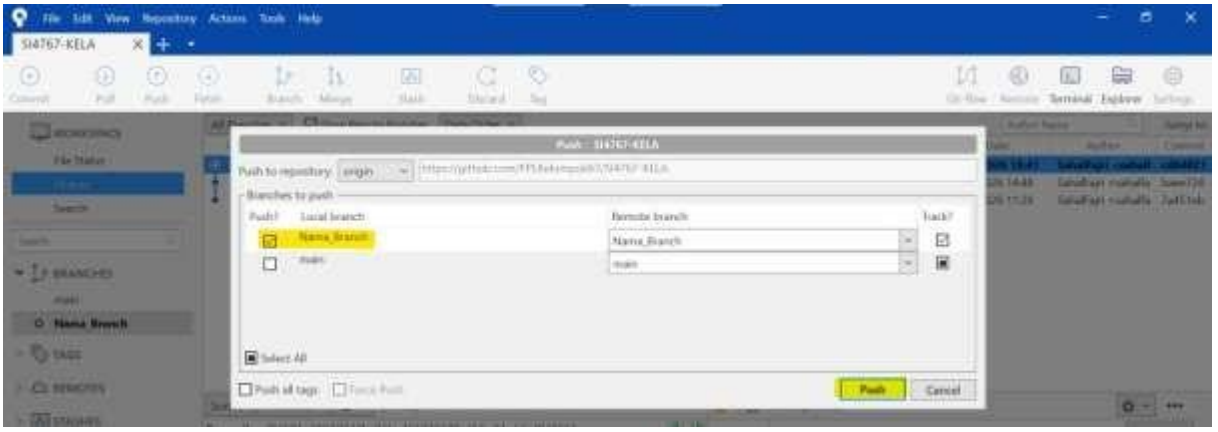
3. Berikut merupakan tampilan branch yang baru dibuat. Terdapat 2 branch yaitu branch master/main dan branch yang baru saja dibuat.



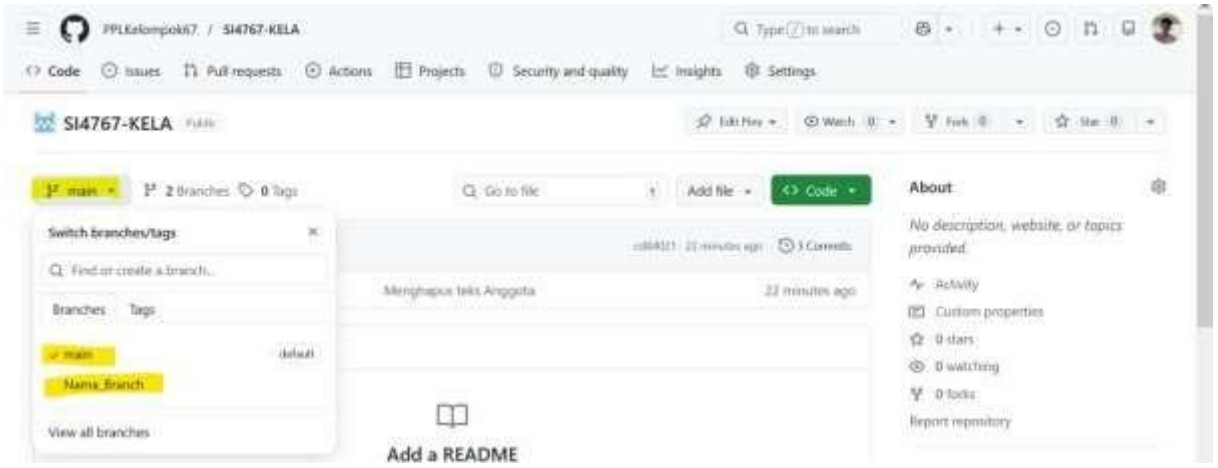
4. Kemudian push branch baru ke Github. Klik kanan pada nama branchnya, lalu pilih “push to”, lalu pilih “origin”.



5. Kemudian akan tampil dialog box seperti dibawah ini. Lalu checklist branch yang ingin di push, lalu klik tombol push.



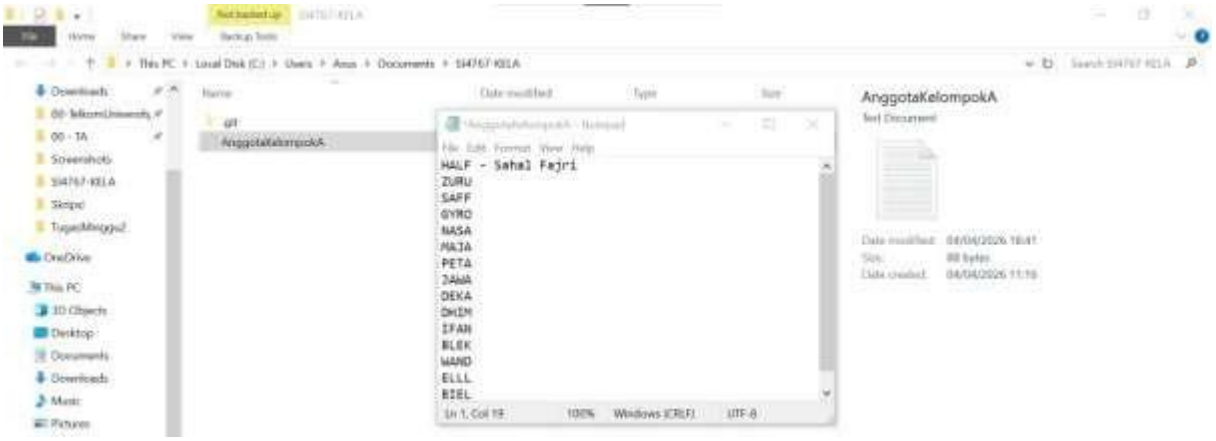
6. Kemudian check pada Github, dan sekarang terdapat dua branch yaitu branch master/main dan branch yang baru saja di push.



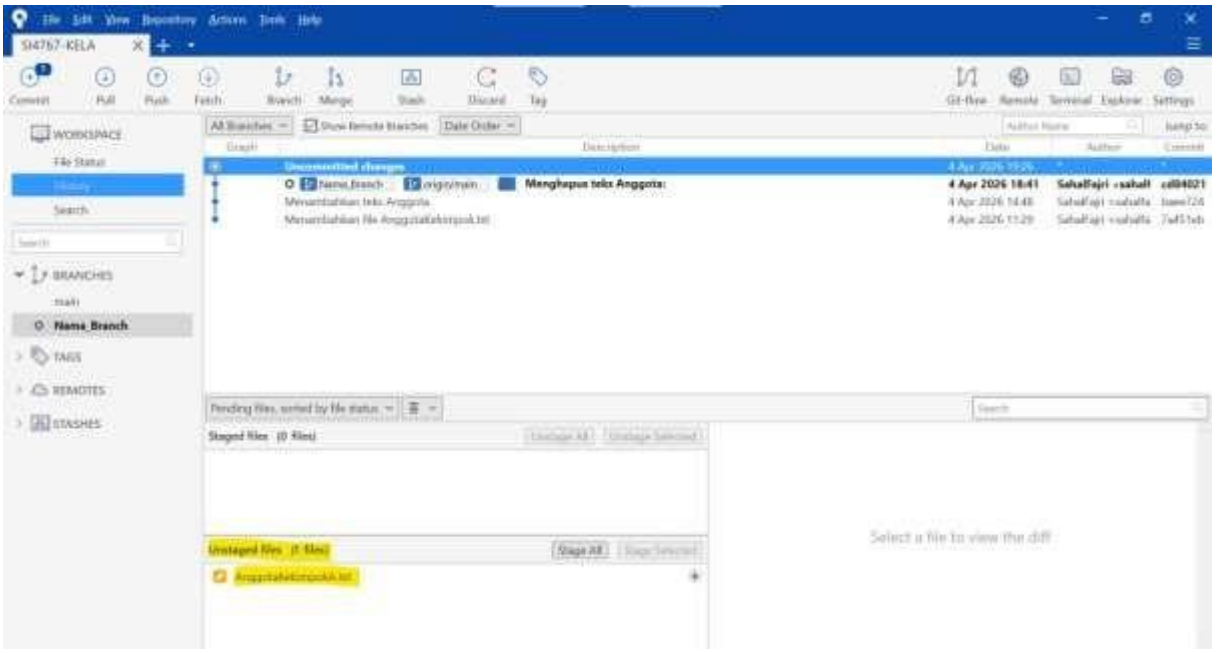
7. Kembali ke SourceTree. Pastikan current branch aktif adalah branch yang baru saja dibuat. Klik “Explorer” pada menu kanan atas untuk ke direktori local.



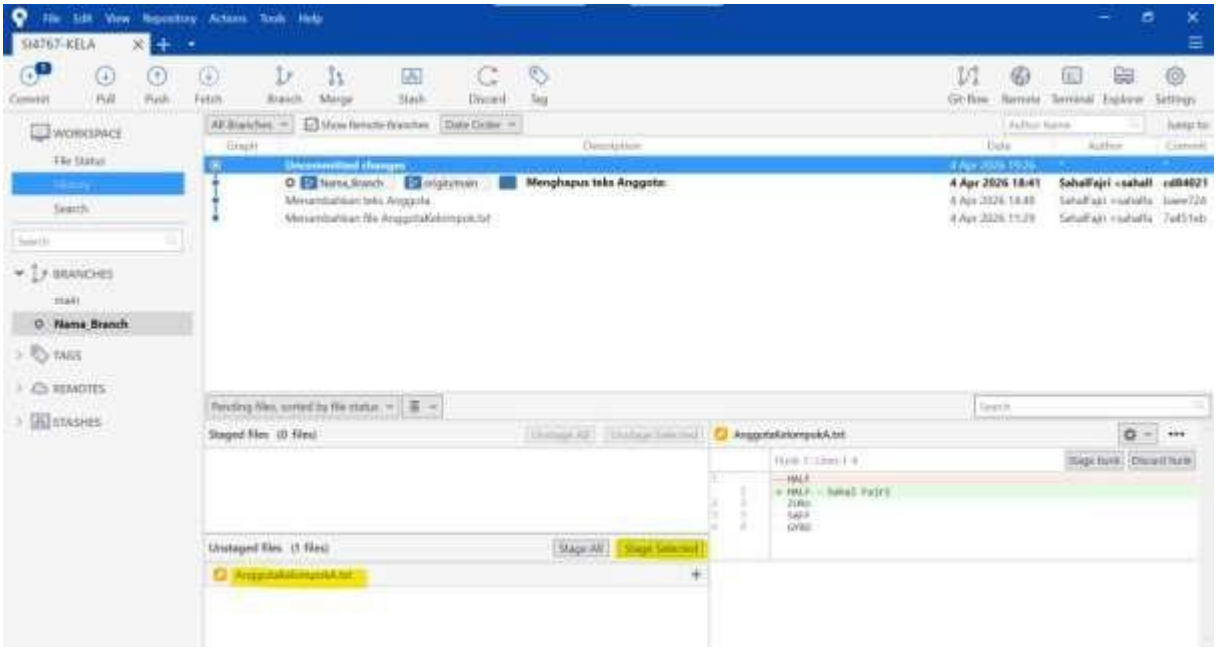
8. Lalu akan di arahkan ke direktori *repository* local. Sudah terdapat file .txt (asumsikan file tersebut sudah dibuat dan di push oleh PM sebelumnya). Kemudian buka file tersebut dan lakukan perubahan (penambahan text atau paragraf). Lalu save file tersebut.



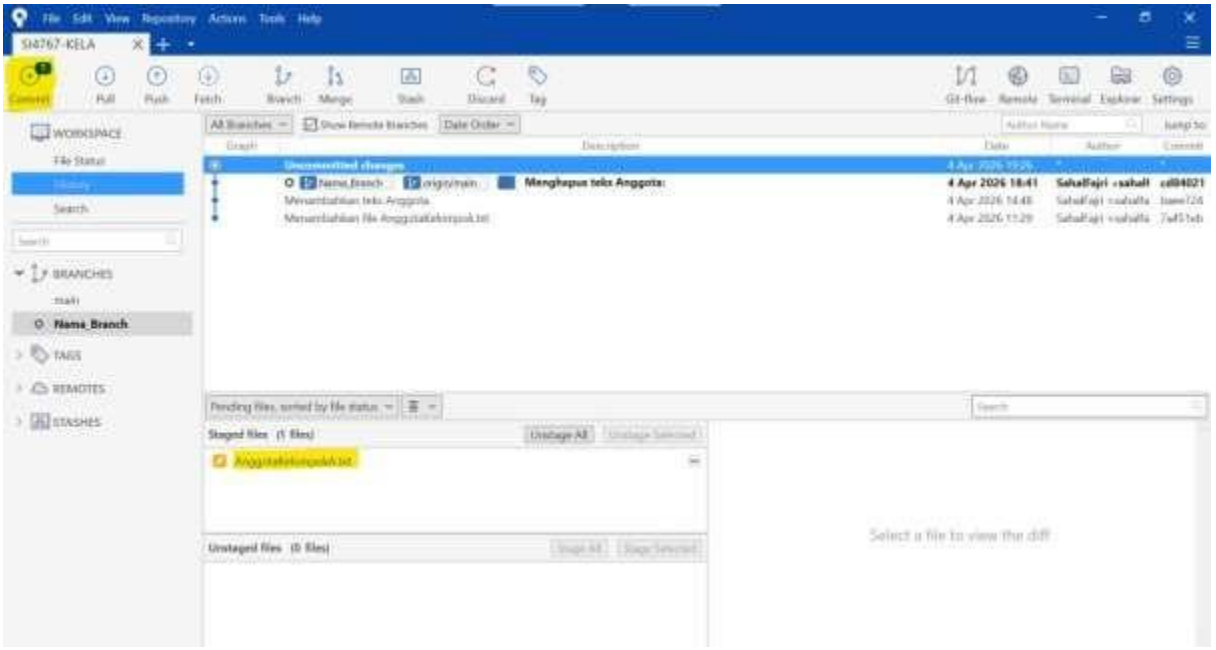
9. Lalu kembali ke SourceTree dan tekan F5 untuk melakukan refresh, maka akan menampilkan notice “Uncommitted changes” pada history graph, dan terdapat file apa saja yang mengalami perubahan pada bagian “Unstaged Files”.



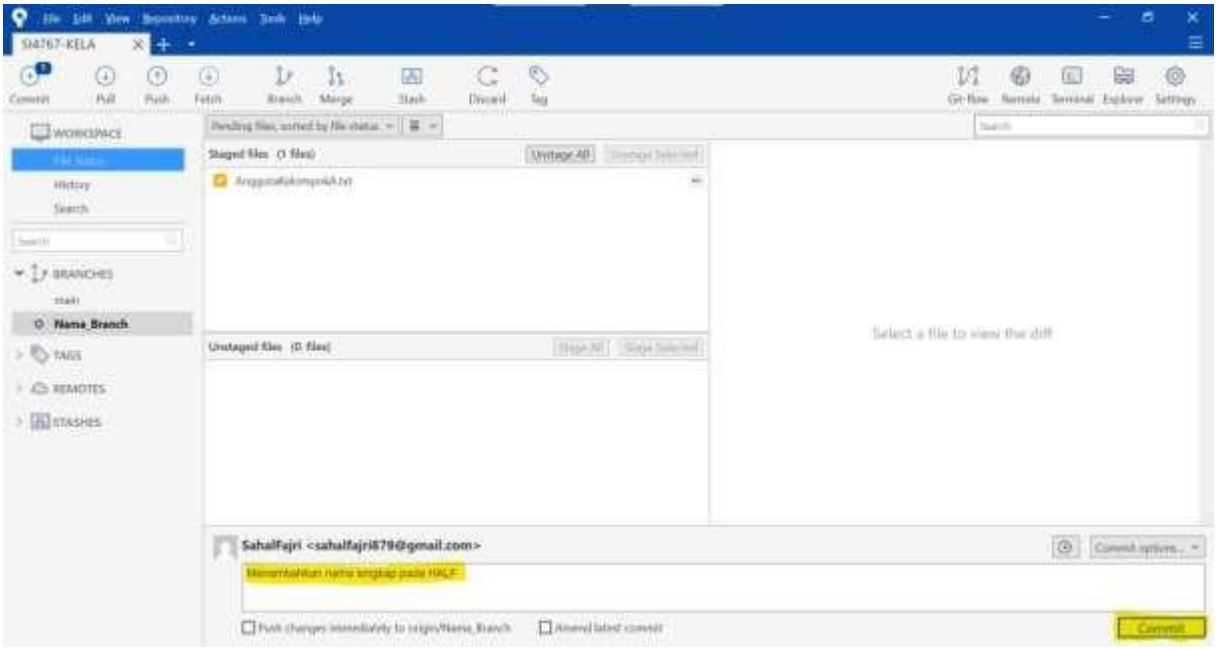
10. Untuk melakukan commit file yang mengalami perubahan ke branch, pada bagian “Unstaged Files” klik file yang akan di commit. Lalu klik “Stage Selected”



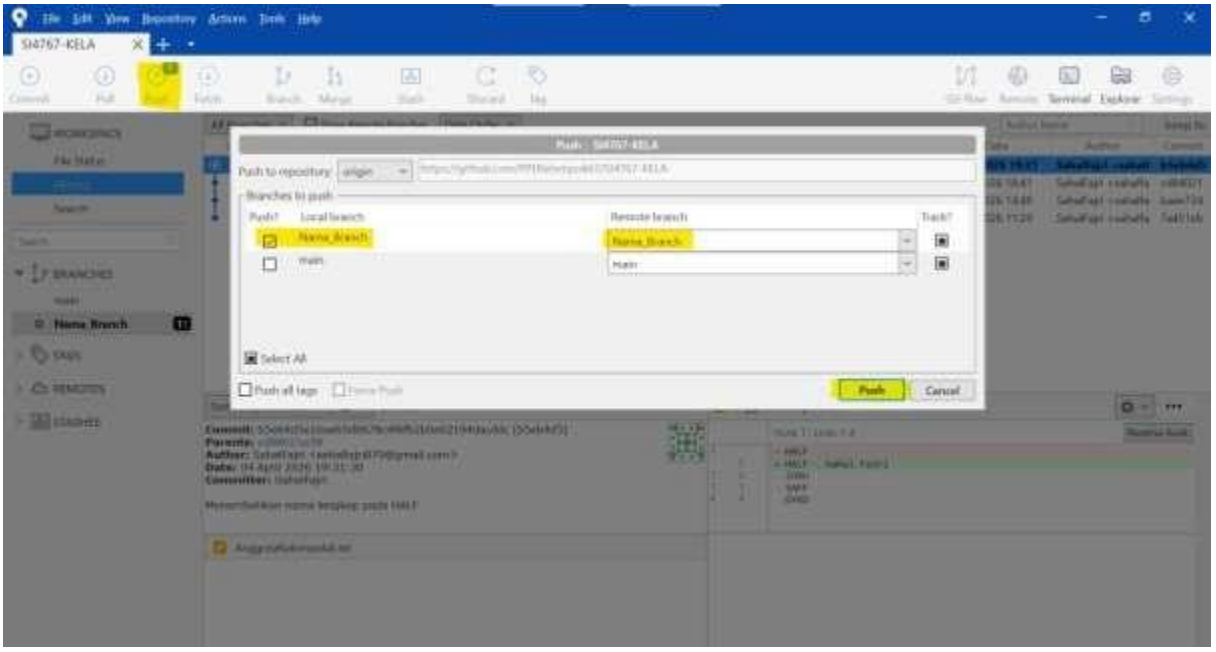
11. Kemudian file tersebut akan pindah ke bagian “Staged Files”. Lalu klik tombol “Commit” pada menu kiri atas.



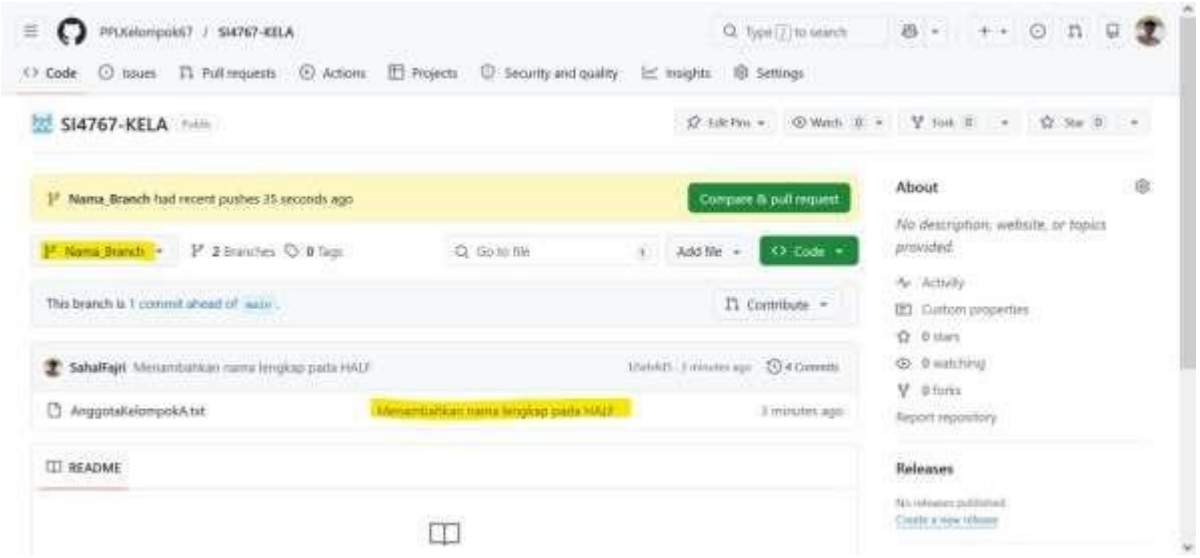
12. Maka akan muncul sebuah box inputan untuk memasukan pesan commit. Kemudian masukkan pesan commit, lalu klik button commit.



13. Selanjutnya melakukan push untuk mengupload file tersebut ke remote *repository* Github. Klik tombol “Push” pada menu atas, lalu pilih remote branch tujuan, lalu klik tombol push.

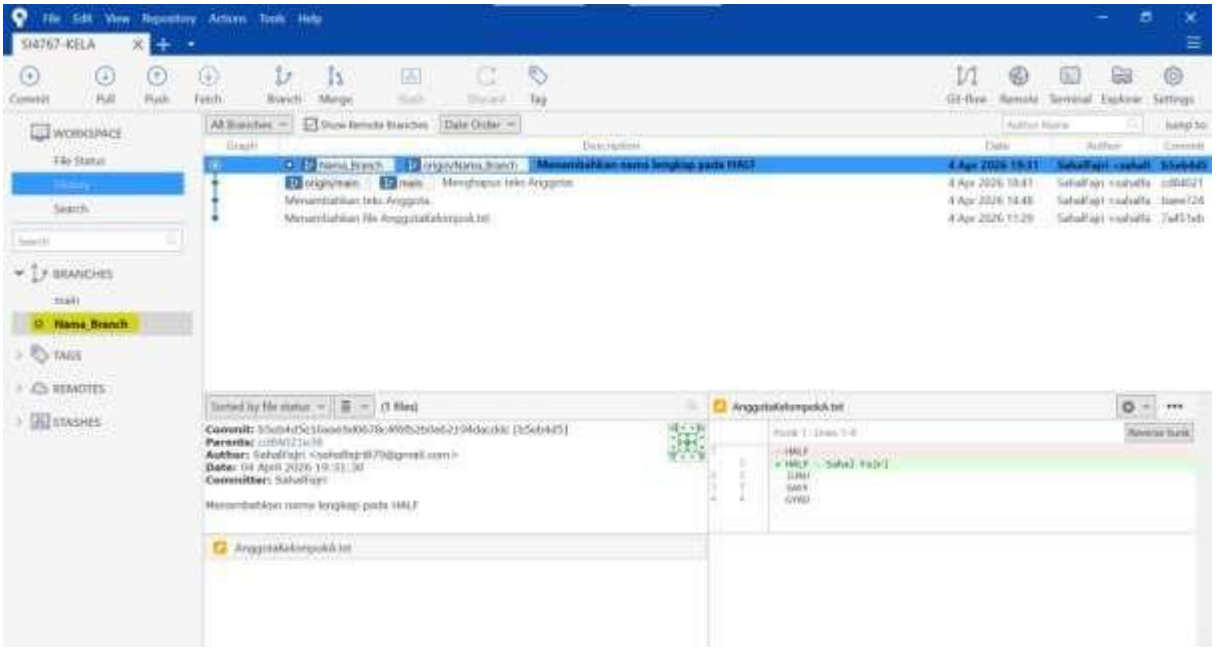


14. Kemudian check di remote *repository* github , lalu pilih branch, akan tampil seperti di bawah ini.



Perubahan file yang di push tadi hanya terjadi pada branch yang baru dibuat yaitu Nama_Branch. Hal itu terjadi karena kita membuat perubahan, commit, dan push di branch yang aktif yaitu Nama_Branch.

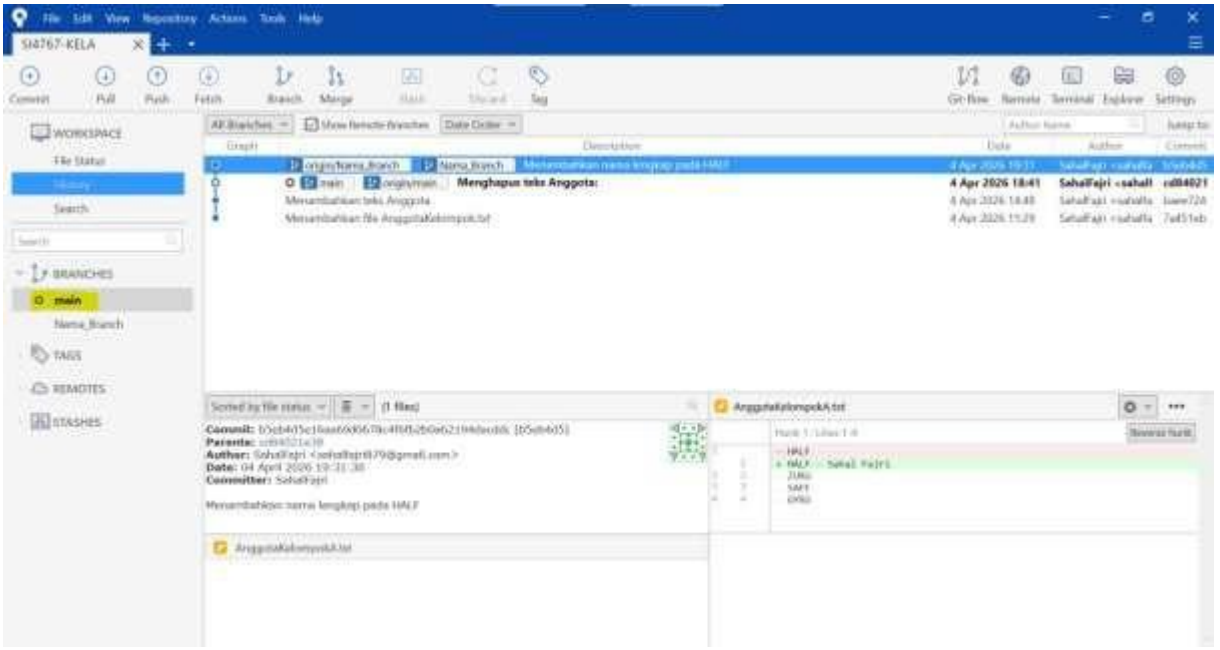
15. Kembali ke SourceTree, teman-teman dapat memilih current branch (yang aktif) dengan cara mengklik dua kali nama branch di bawah menu samping “Branches”. Untuk mengetahui branch mana yang aktif, teman-teman bisa tahu dari tampilan fontnya yang tebal.



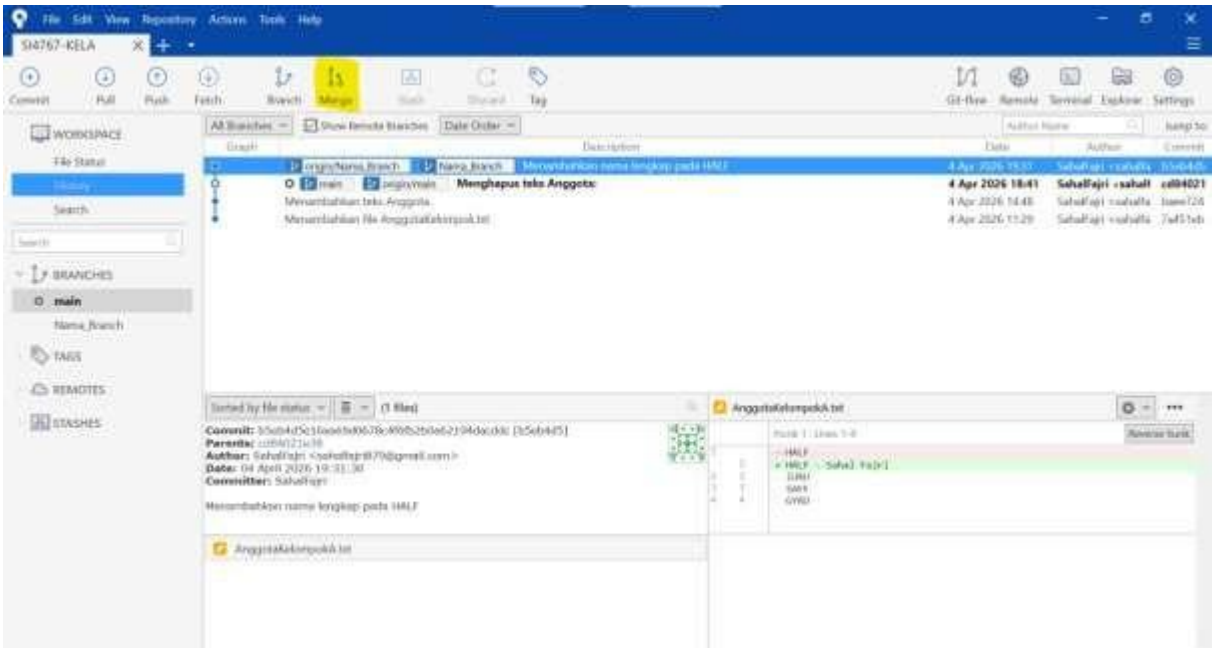
c. Merging File Perubahan dari Branch

Kita coba berpindah ke branch master/main lalu menggabungkan branch Nama_Branch ke dalamnya.

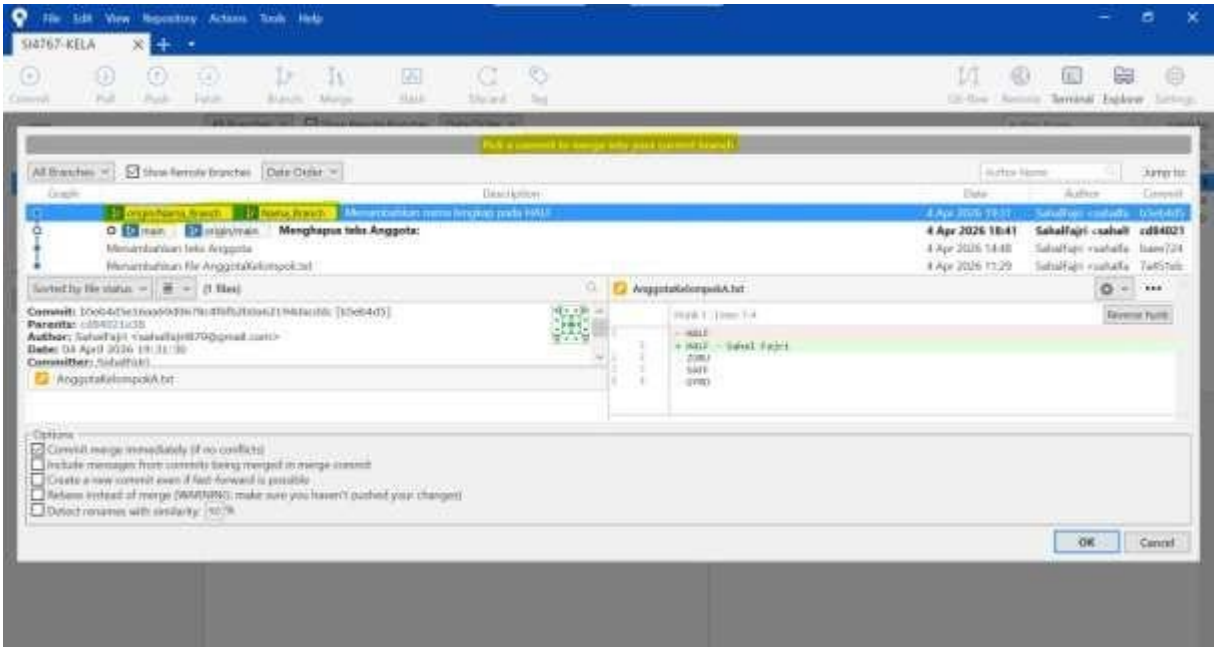
1. Pertama, switch ke branch master/main.



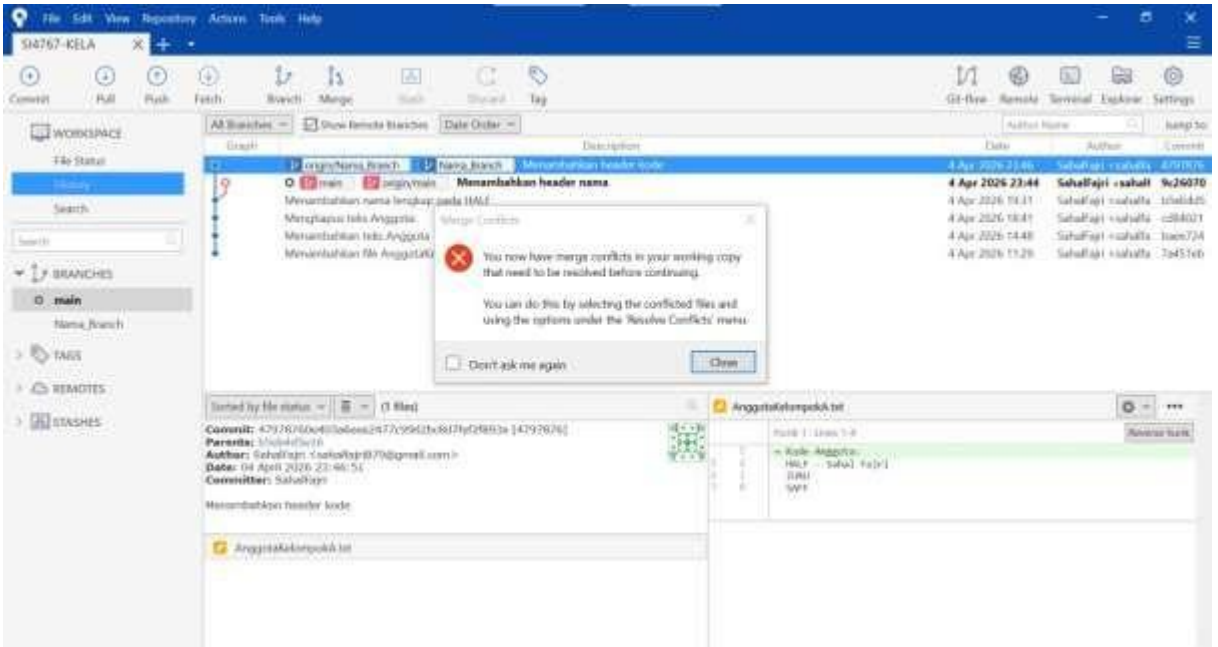
2. Klik tombol “Merge”



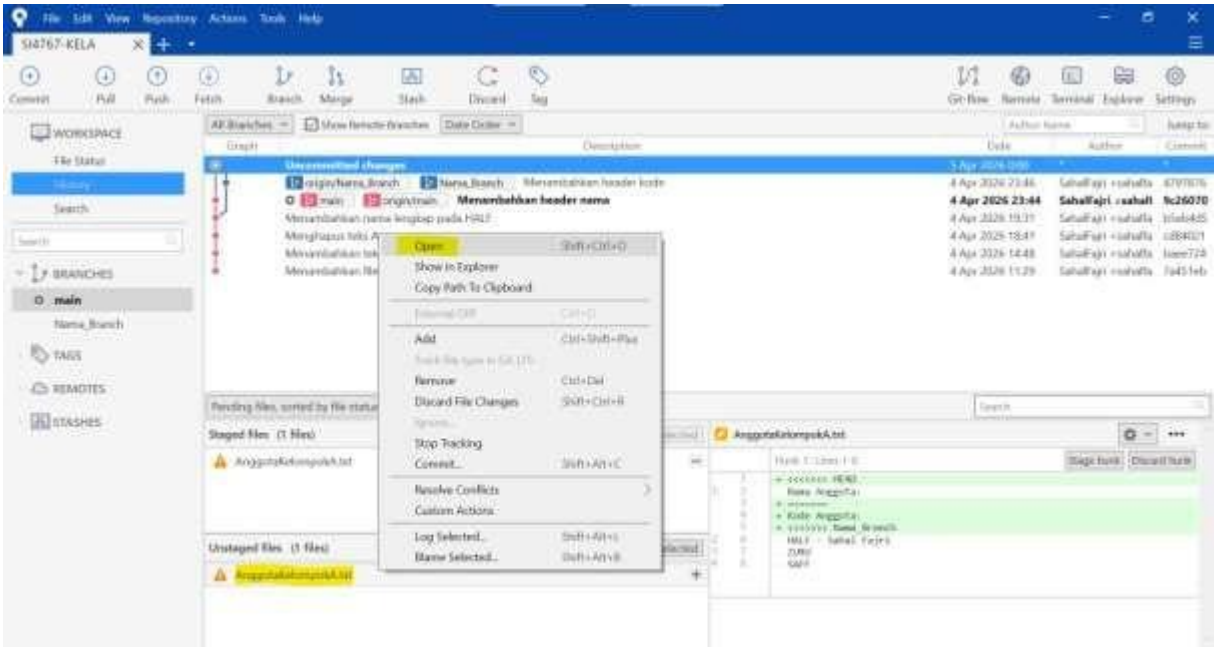
3. Dari popup yang muncul, pastikan di daftar commit, branch yang ingin digabungkan disorot. Itu menunjukkan bahwa kita ingin menambahkan commit dari branch ini ke branch utama /master atau /main. Kemudian klik OK.



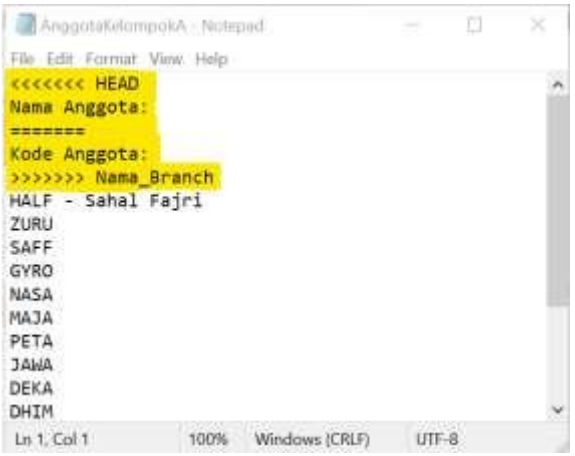
4. Jika muncul pesan seperti dibawah, itu menandakan terjadinya conflict. Tidak selamanya proses merging berjalan mulus tanpa masalah. Kadang kita mengalami conflict karena dalam kedua branch yang akan kita gabungkan ada satu atau beberapa file yang isinya berbeda di branch yang satu dengan yang lain. Hal ini akan lebih sering kita alami ketika kita berkolaborasi dengan orang lain menggunakan shared repository. Di sini kita berada dalam situasi yang disebut merge conflict karena file .txt di branch master/main dan file .txt di branch Nama_Branch, memiliki perbedaan isi tepat pada baris yang sama. Klik Close.



5. Untuk melihat baris dan file yang menyebabkan konflik, klik kanan pada file yang terjadi konflik pada bagian Unstaged Files, lalu klik Open.



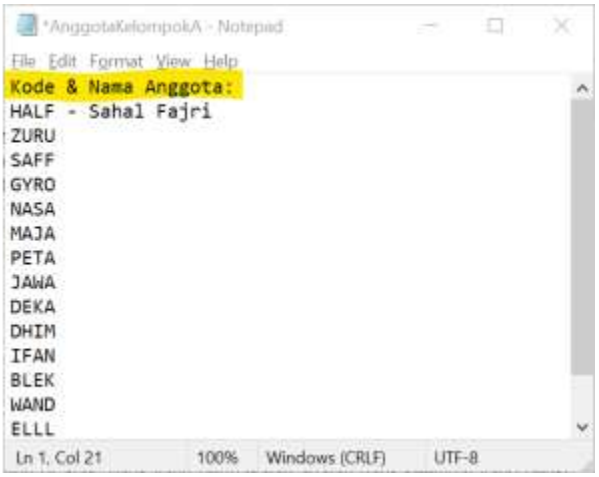
6. Maka akan terbuka file yang terjadi konflik. Baris yang mengakibatkan konflik ditandai dengan deretan karakter <<<<< dan >>>>> di dalam file yang mengalami konflik, dalam contoh di atas, file .txt.



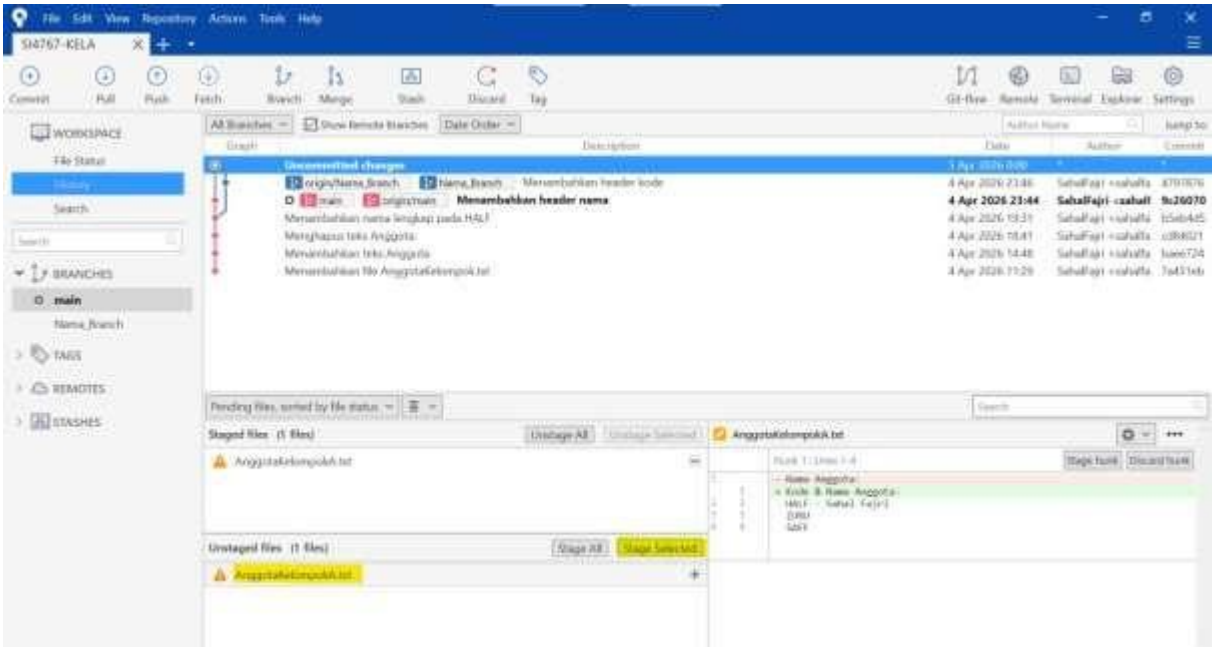
Dalam contoh di atas, baris yang bermasalah adalah baris keempat yang berisi teks "Nama Anggota" (main) dan "Kode Anggota" (branch Nama_Branch).

Untuk menyelesaikan conflict, ada dua opsi yang kita miliki yaitu menghapus salah satu baris atau menggabungkan keduanya. Opsi manapun yang kita pilih, kita harus menghapus baris yang berisi tanda <<<< dan >>>> serta baris yang berisi ===== lalu melakukan commit.

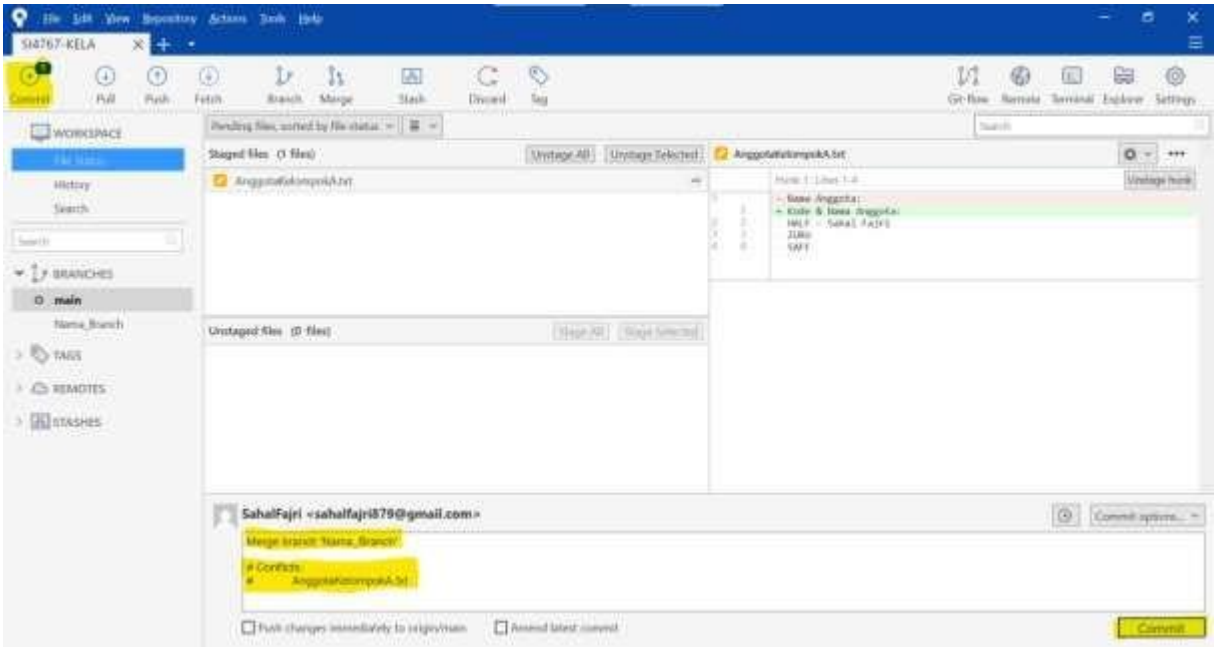
Kita gabungkan kedua teks tersebut dengan mengubah isi file .txt menjadi seperti berikut:



7. Kemudian lakukan commit. Kembali ke SourceTree, pada bagian Unstaged Files, klik file .txt, lalu klik tombol Stage Selected.



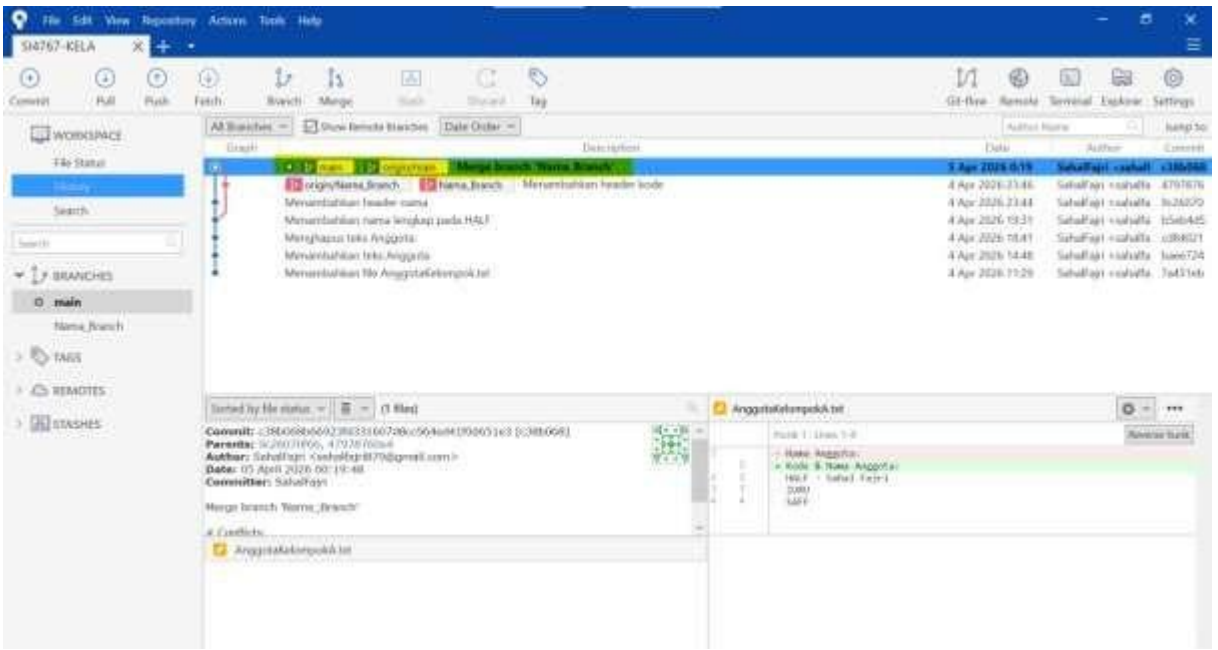
8. Klik tombol Commit pada menu kiri atas. Maka akan otomatis pesan commitnya sudah dibuatkan, lalu klik tombol commit.



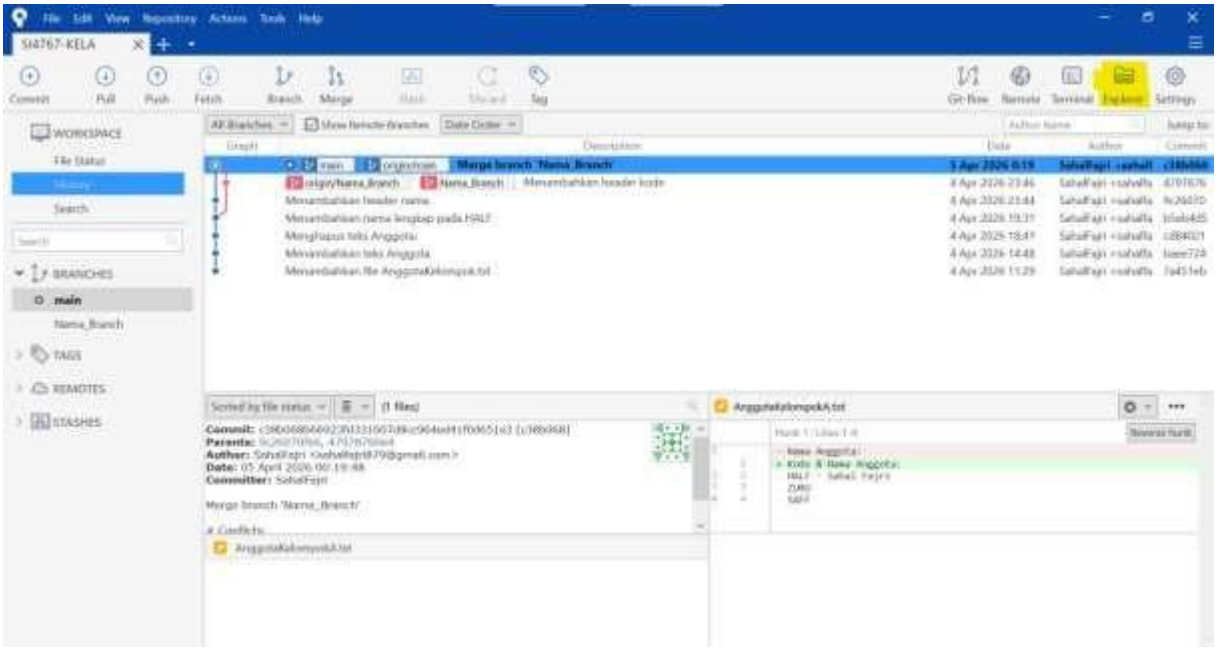
9. Kemudian melakukan push. Klik tombol push pada menu atas.



10. Jika proses push berhasil, maka akan menampilkan notif di history graph “Merge branch ‘Nama_Branch”.



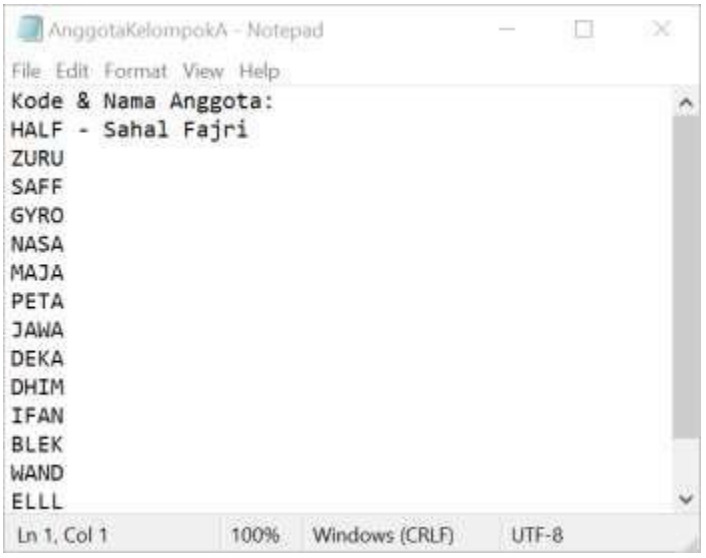
11. Untuk mengecek apakah berhasil melakukan merge antara branch Nama_Branch dan branch master, klik Explorer.



12. Maka di arahkan ke direktori lokal, buka file .txt.



File Project.txt yang dikerjakan secara bersamaan pada dua branch yang berbeda, berhasil digabungkan.

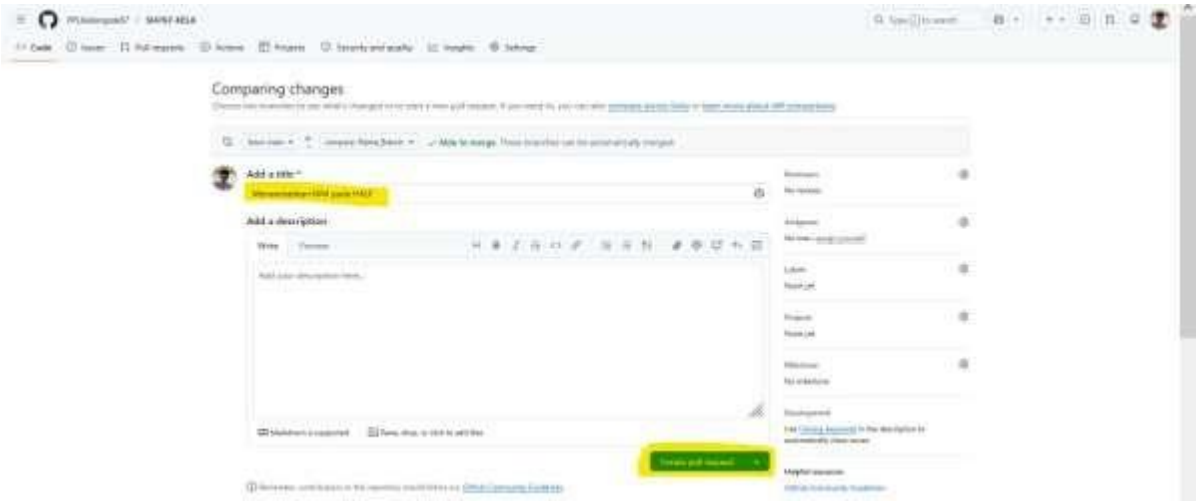


d. Membuat Pull Request

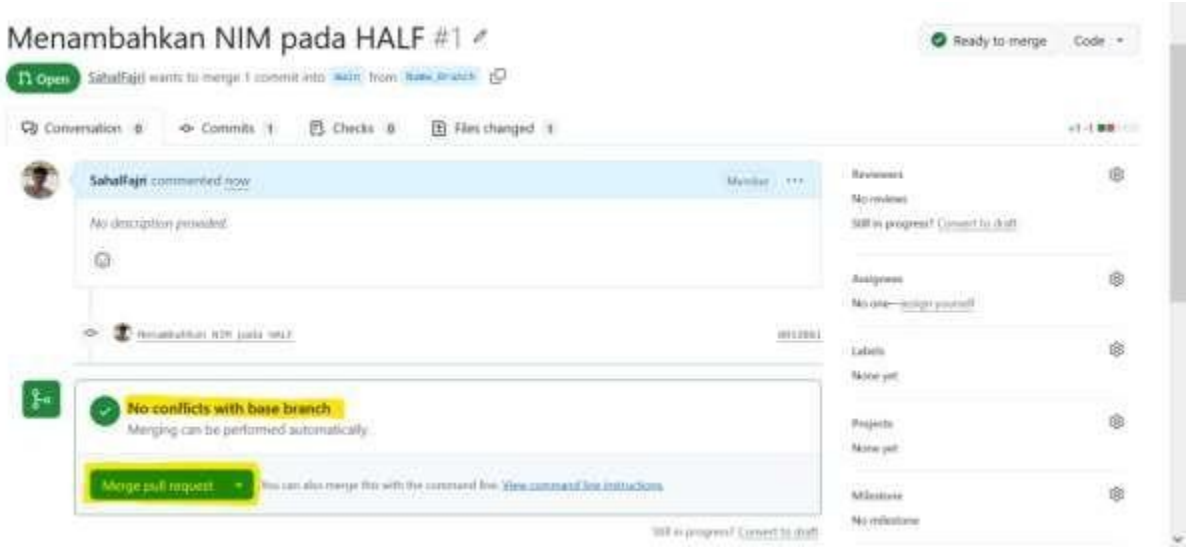
1. Setelah push dilakukan, buka kembali repository GitHub di browser. Maka akan terdapat tombol seperti di bawah ini.



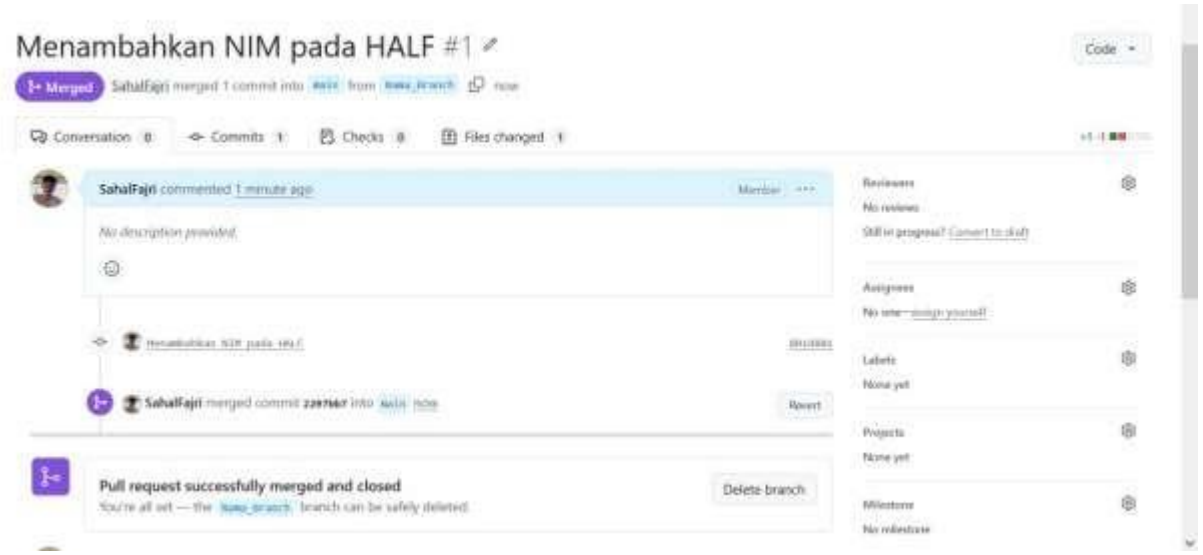
2. Silakan klik tombol Compare & pull request tersebut, lalu akan di redirect ke halaman Open a pull request. Pada halaman ini silakan masukkan judul pull request dan deskripsi (opsional), setelah itu silahkan klik Create pull request.



3. Setelah itu, GitHub akan melakukan komparasi. Apakah ada kode yang bentrok atau tidak? Kalau tidak ada yang bentrok biasanya akan muncul tulisan “This branch has no conflicts with the base branch”. Kemudian klik tombol “Merge pull request”.



4. Lalu melakukan merge, klik tombol “confirm merge” dan jika berhasil akan ada tulisan “Merged” berwarna ungu.

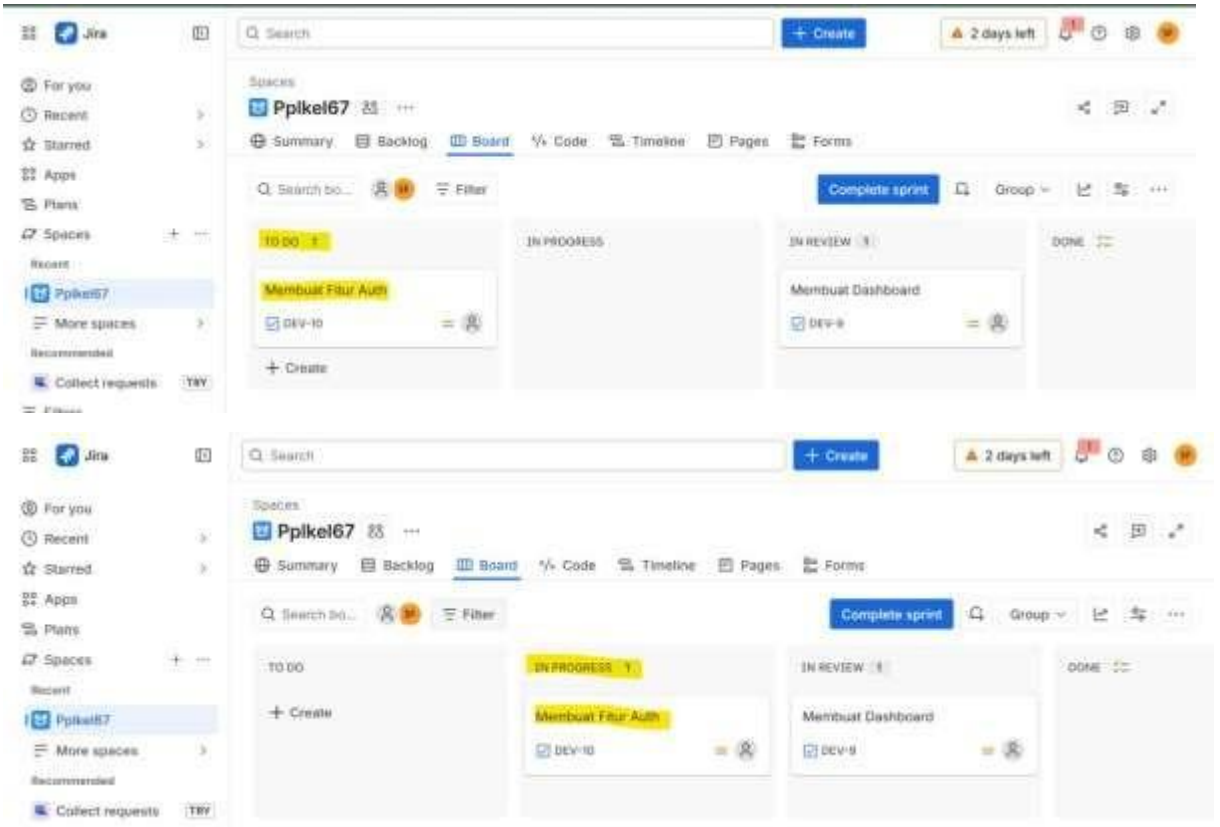


5. Maka pengerjaan yang dilakukan di branch telah berhasil digabungkan ke branch utama (main/master).

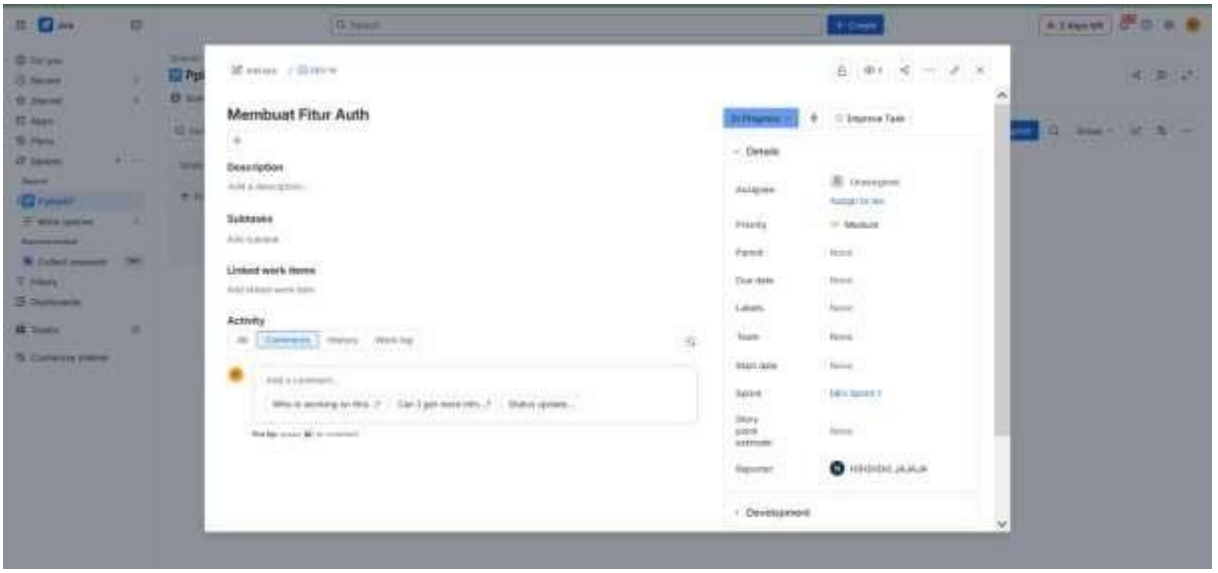
3. Integrasi dengan Jira menggunakan Issue Key

a. Commit dengan Issue

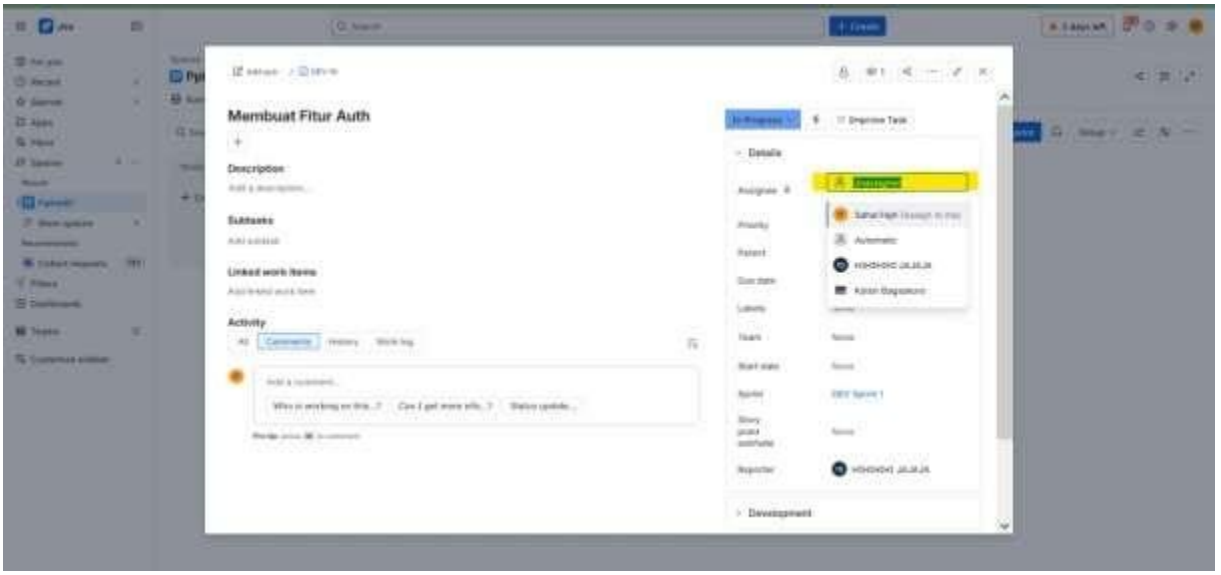
1. Drag and drop product backlog yang ingin dilakukan dari “TO DO” ke bagian “IN PROGRESS”.

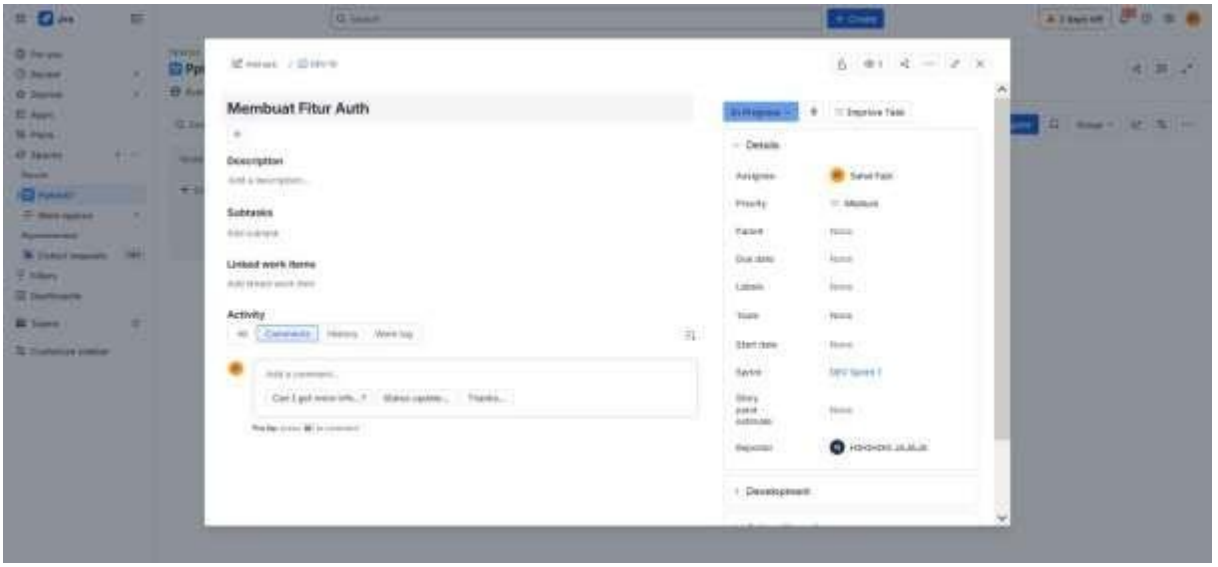


2. Klik backlog “IN PROGRESS” untuk melihat detail backlog.

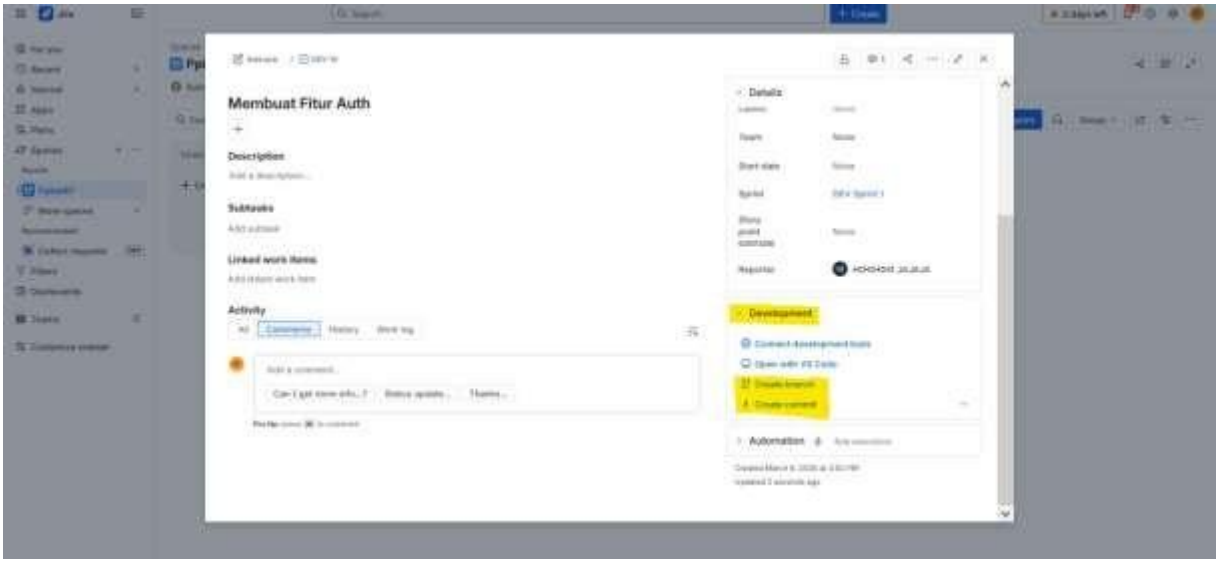


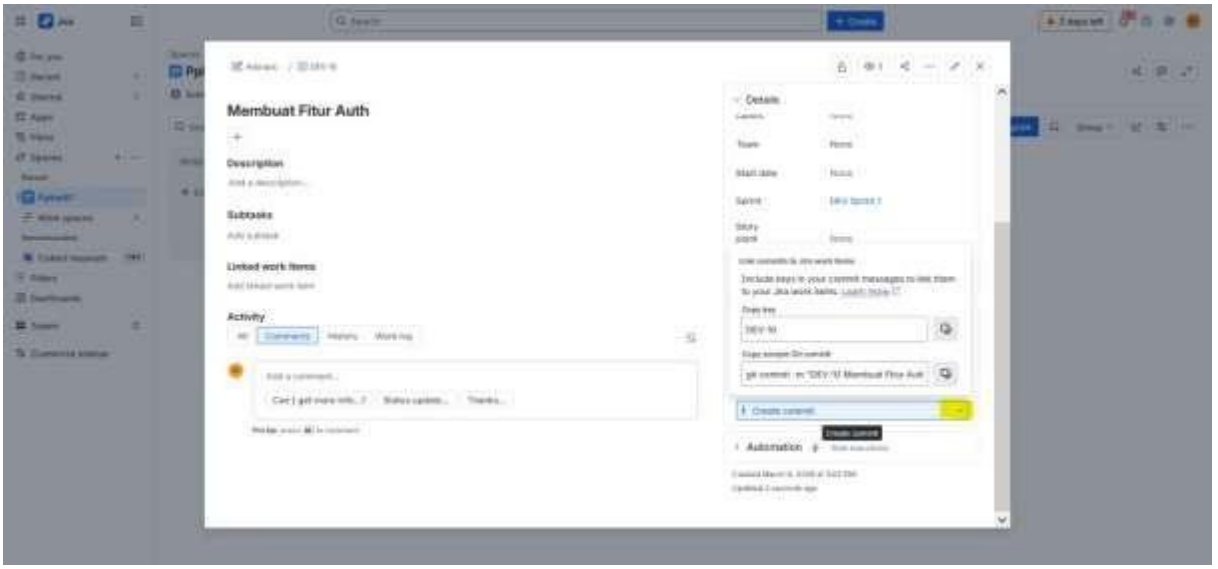
3. Dari detail backlog ini, seorang project manager dapat menetapkan assignment kepada anggota tim dengan merubah bagian Assigned di bawah Details.





4. Di detail backlog ini, seorang project manager juga dapat memantau progress development perangkat lunak dengan melihat bagian Development di bawah Details. DI bagian development ini, kita dapat melihat branch dan commit mana saja yang berkaitan dengan backlog ini sekaligus melihat issue key dari backlog ini.

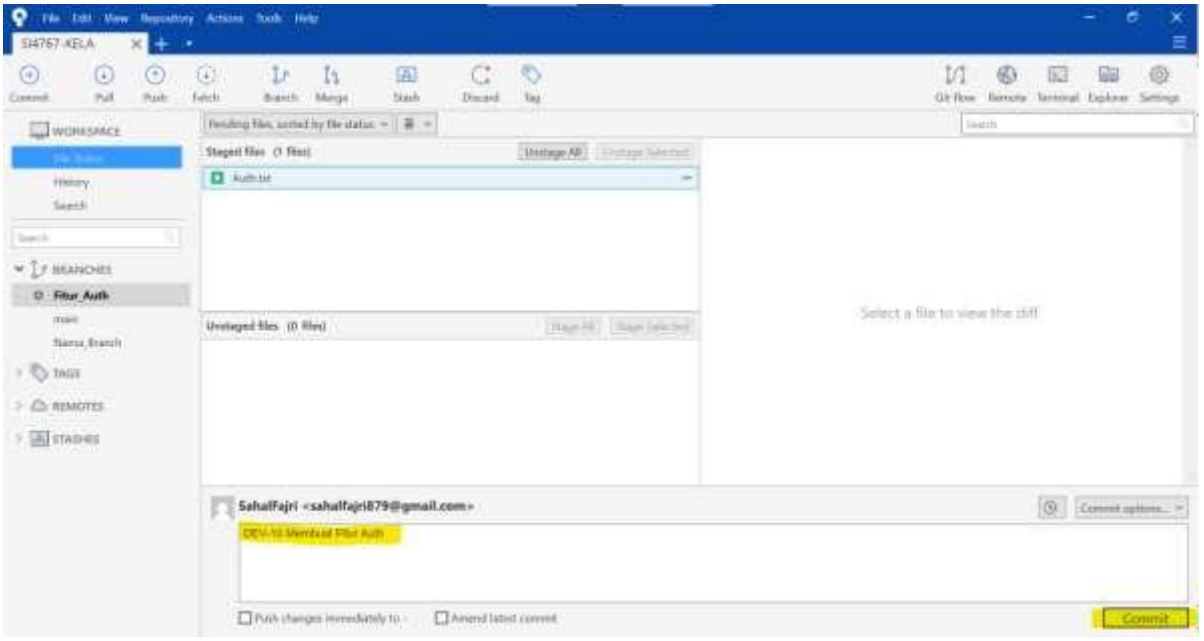




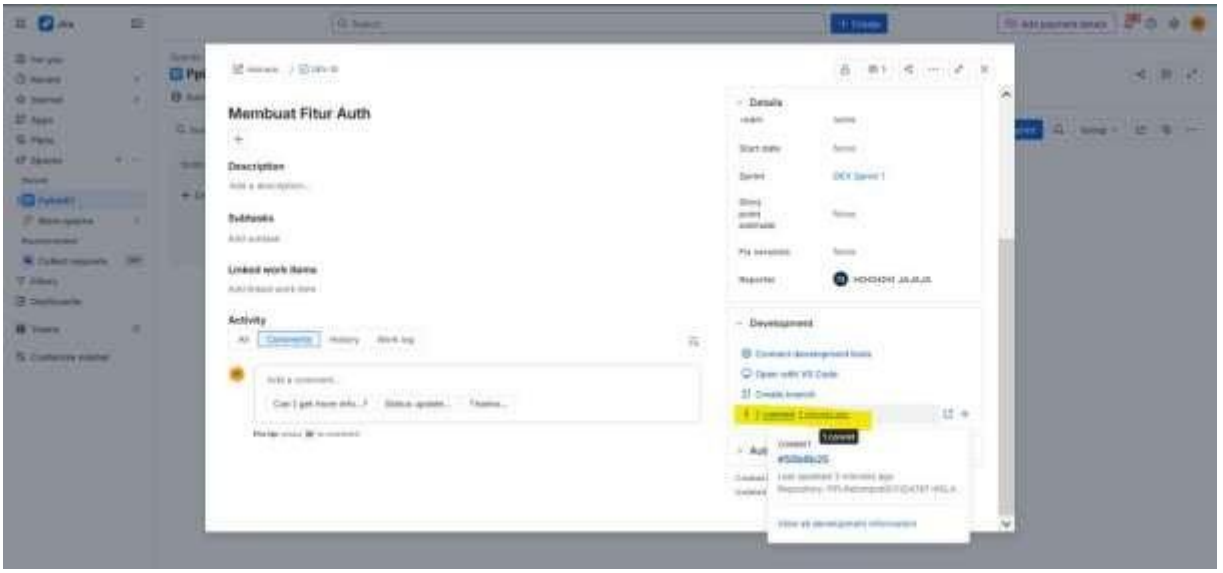
5. Untuk menambahkan commit di Jira, perlu dilakukan perubahan pada repository. Sebagai contoh, buatlah sebuah file .txt baru di dalam folder repository.



6. Setelah itu lakukan commit dengan pesan yang berisi issue key dari backlog. Dalam contoh ini, issue key yang digunakan adalah DEV-10. Setelah itu lakukan push (kamu bisa juga menambahkan pesan dalam commit tersebut).

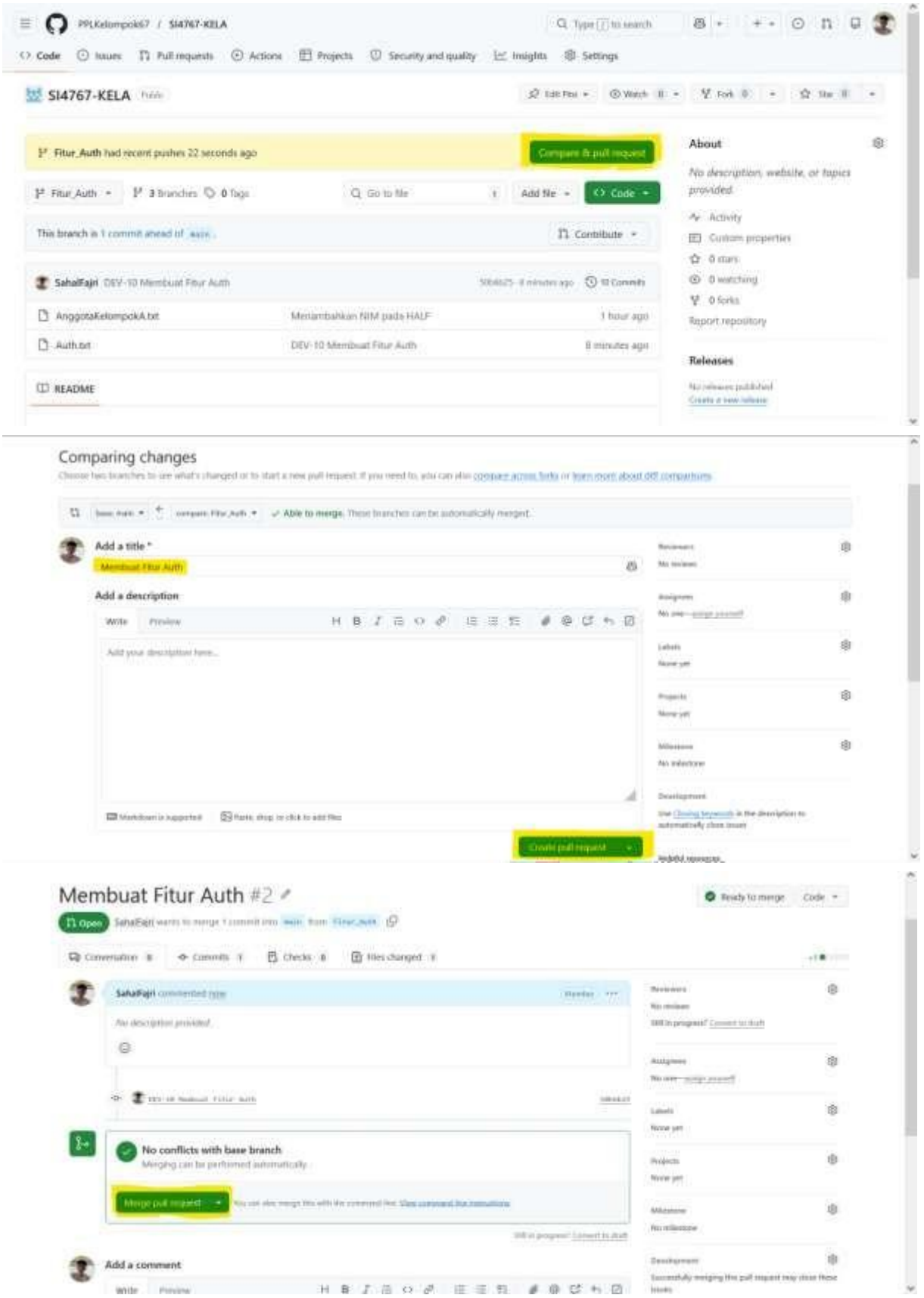


7. Apabila berhasil di push, kita bisa melihat commit tersebut muncul di detail backlog.

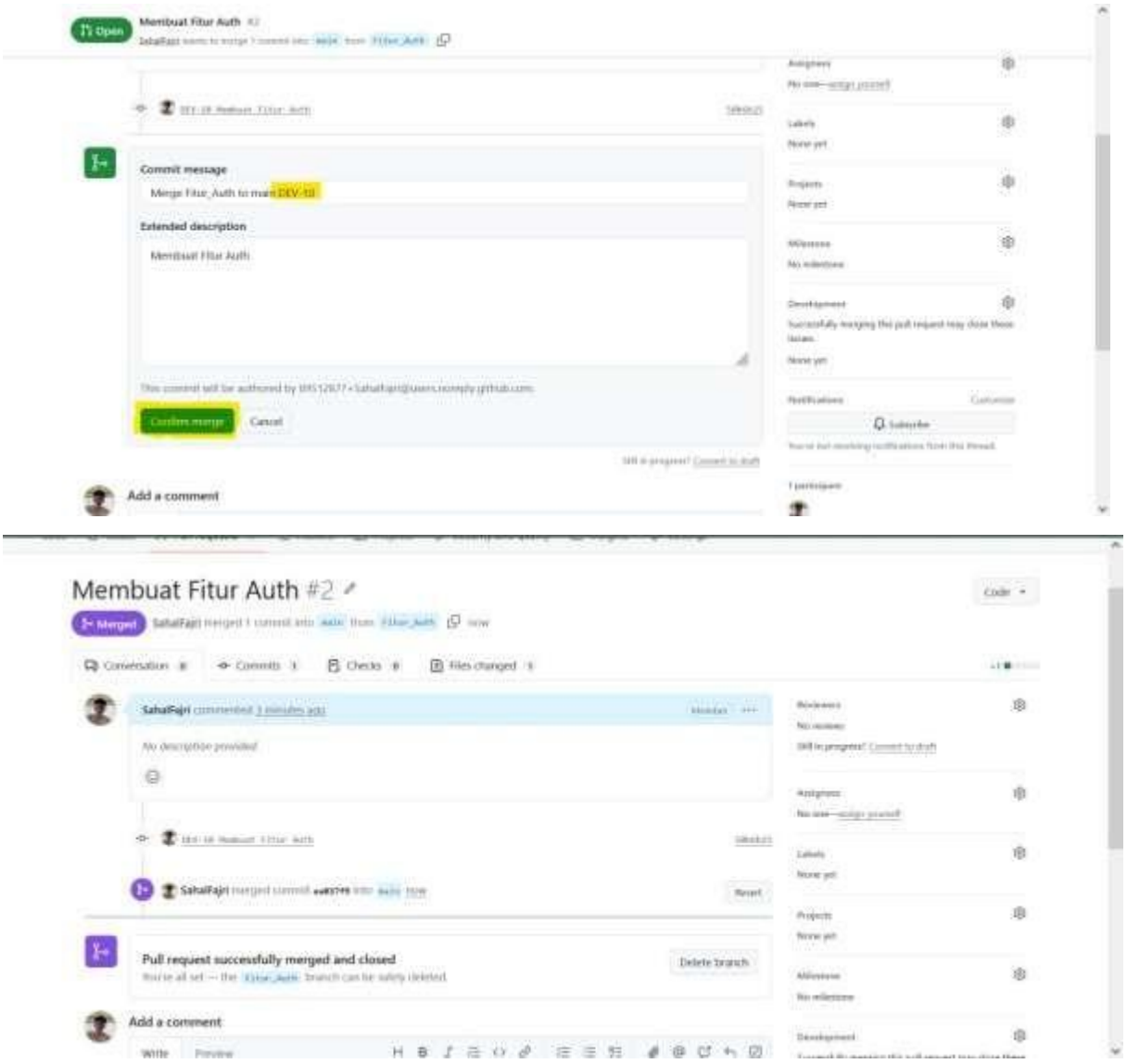


b. Merge & Pull Request dengan Issue

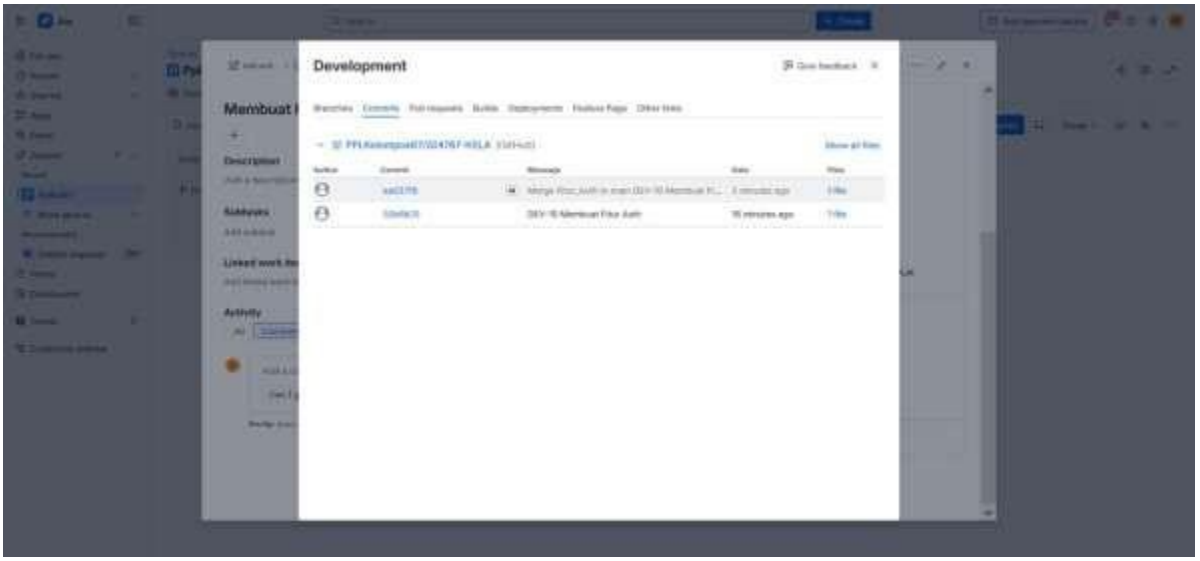
1. Lakukan pull request di Github untuk menggabungkan branch (Fitur_Auth) dengan main, lalu isi pesan commit dengan issue key agar dapat disambungkan dengan Jira.



2. Pada commit message jangan lupa tambahkan issue key sesuai pada backlog.



3. Di Jira, kita dapat melihat perubahan merge yang baru kita lakukan. Terlihat jumlah commit bertambah menjadi 2.



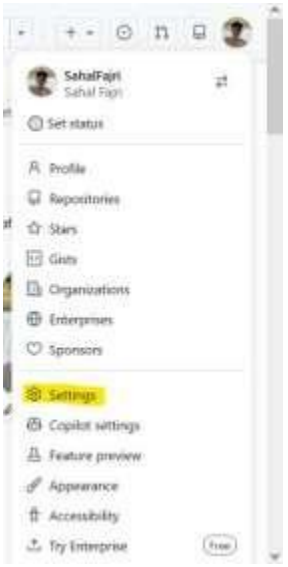
Resolve Error

1. The requested URL returned error: 403

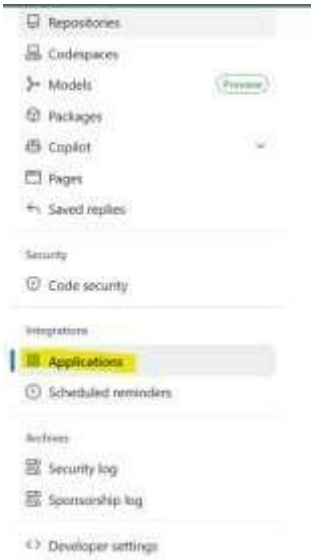


a. Meminta Akses Organization

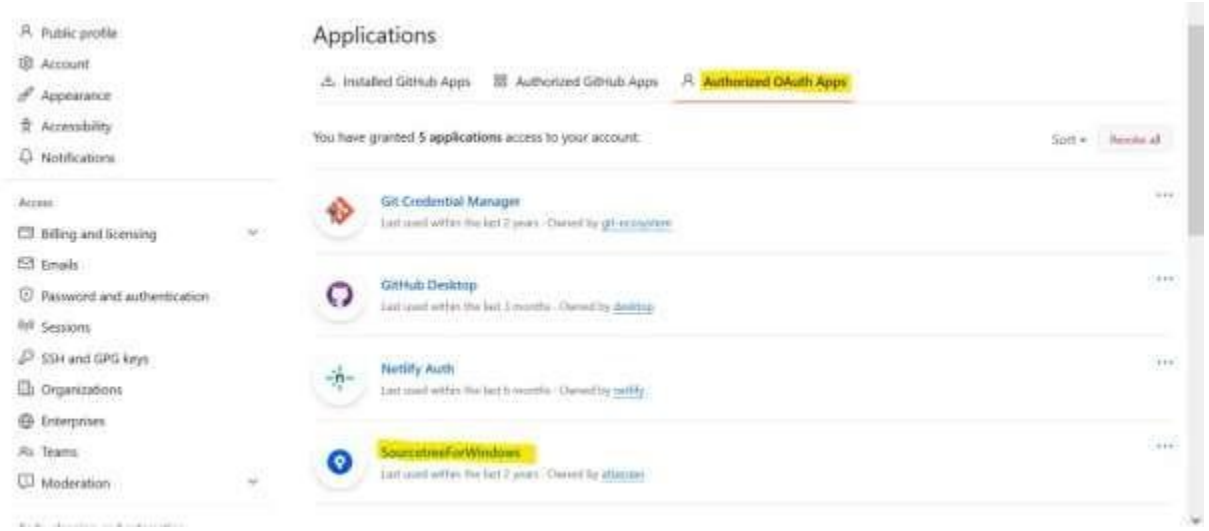
1. Buka **GitHub** > Klik foto profil di pojok kanan atas > **Settings**.



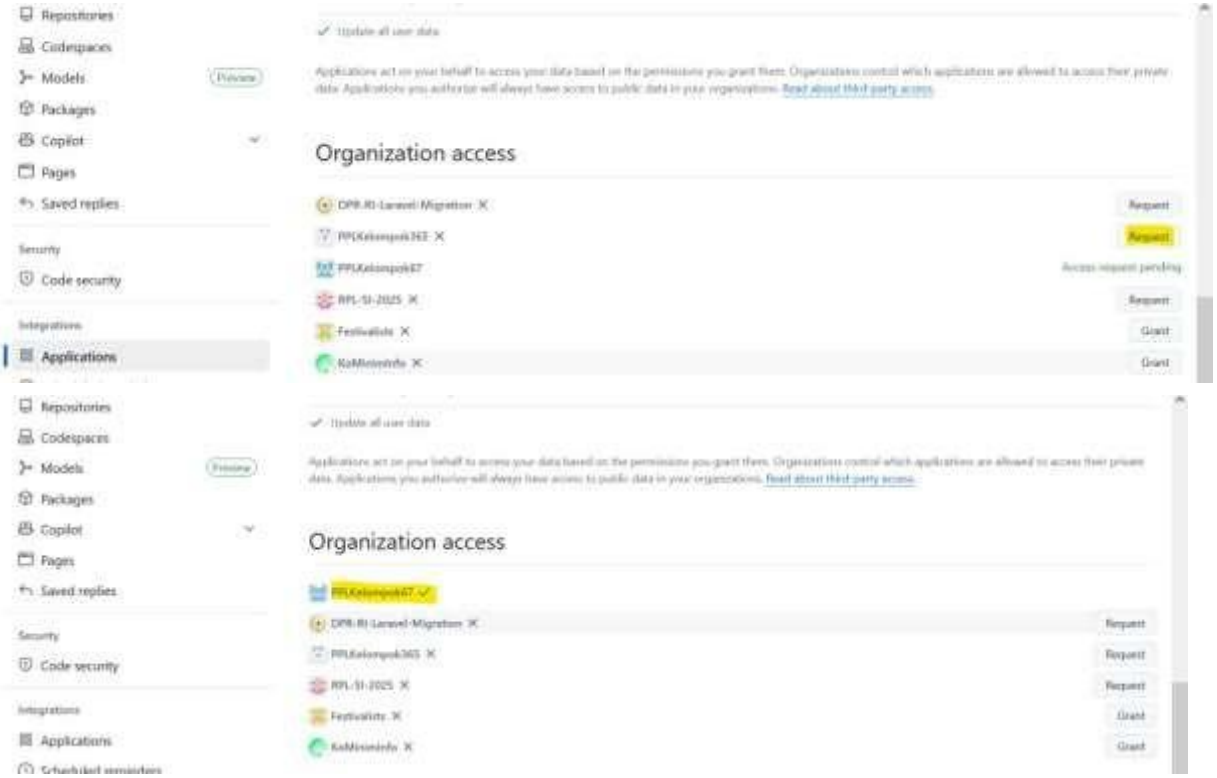
2. Di sidebar kiri, cari menu **Applications** > **Authorized OAuth Apps**.



3. Cari **SourceTree** di daftar aplikasi yang ada, lalu klik namanya.



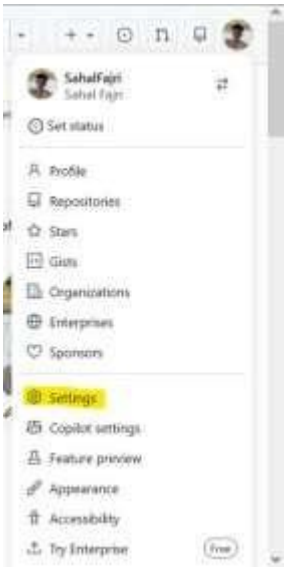
4. Scroll ke bawah ke bagian **Organization access**.
5. Cari organisasi kalian. Jika di sebelahnya ada tombol **Grant/Request**, klik tombol tersebut. Jika ada tanda centang hijau, berarti akses sudah diberikan.



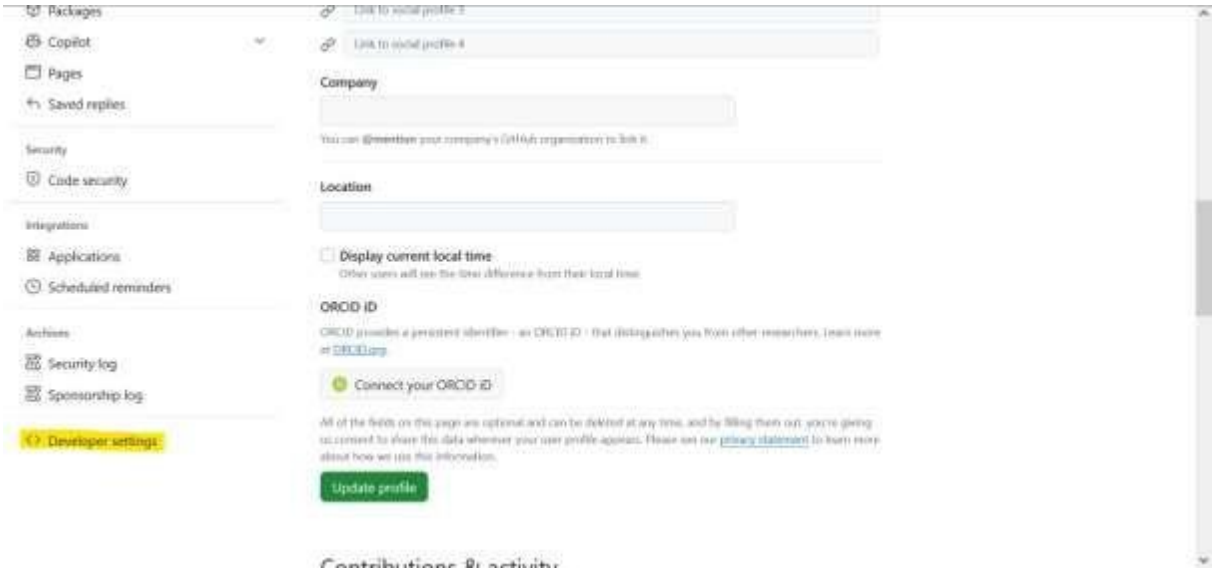
6. Jika sudah di-grant, coba lakukan **Push** lagi di SourceTree.

b. Menggunakan Personal Access Token (Alternative jika a tidak bisa)

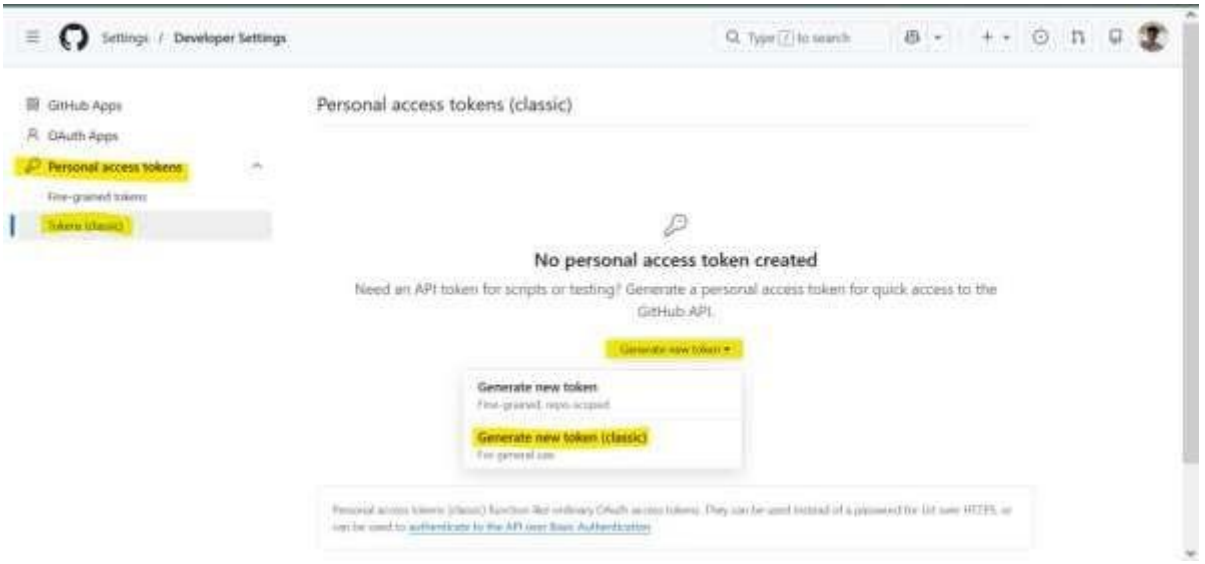
1. Buka **GitHub** > Klik foto profil di pojok kanan atas > **Settings**.



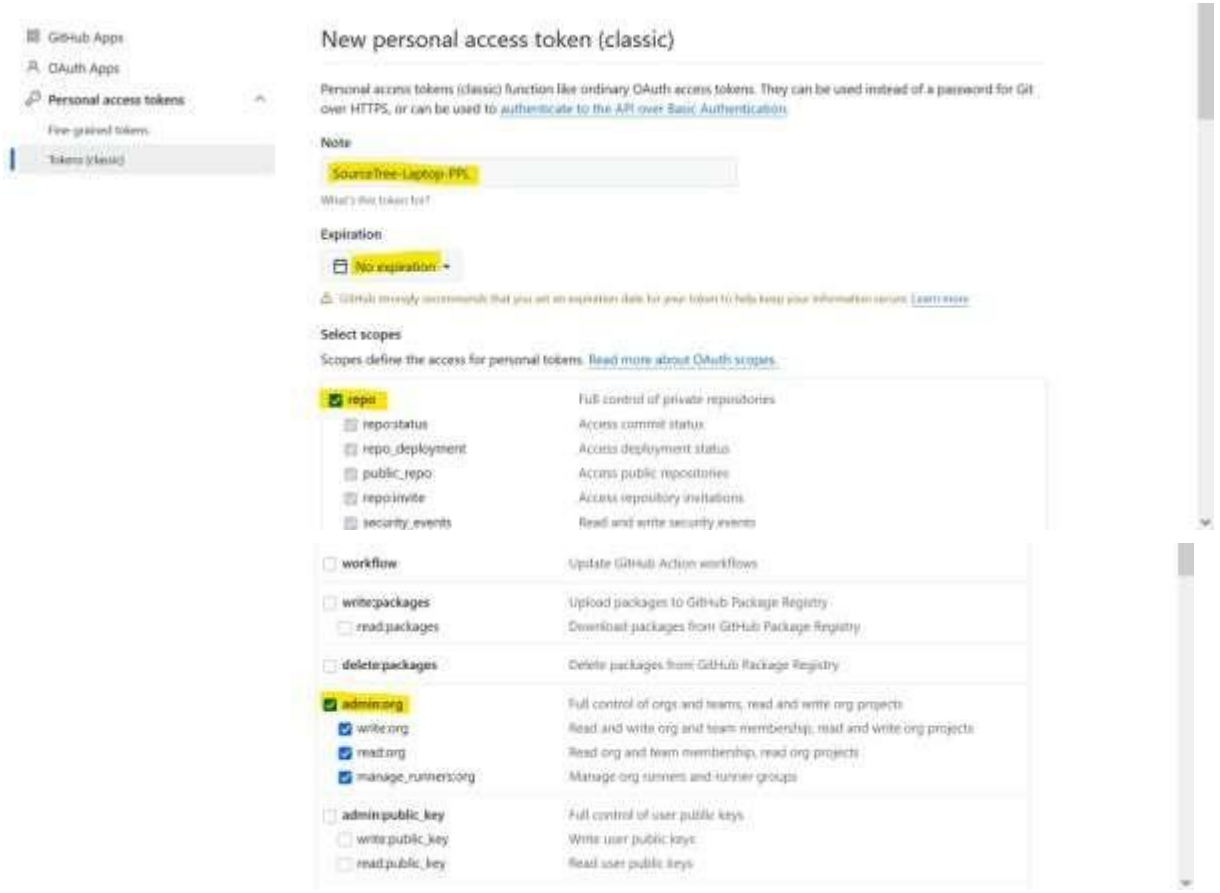
2. Scroll ke paling bawah di sidebar kiri, klik **Developer settings**.

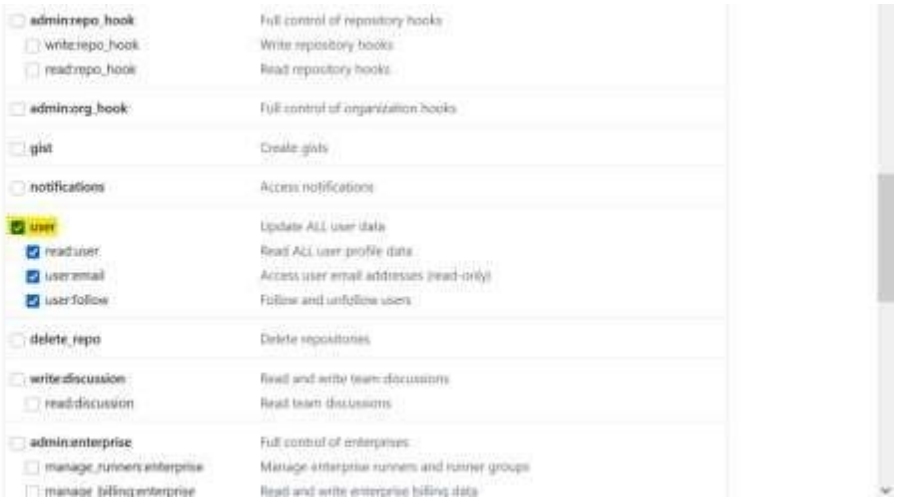


3. Pilih **Personal access tokens** > Klik **Tokens (classic)**.
4. Klik tombol **Generate new token** > **Generate new token (classic)**.

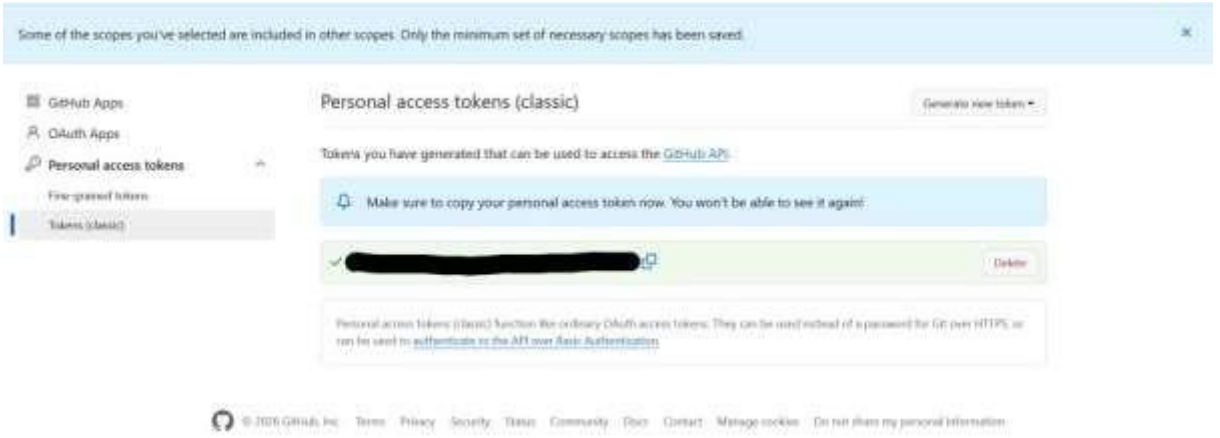


5. Isi **Note** (misal: "SourceTree-Laptop-PPL").
6. Pada **Expiration** ubah menjadi ‘No expiration’
7. **Penting:** Pada bagian **Select scopes**, centang **repo**, **admin:org**, dan **user**

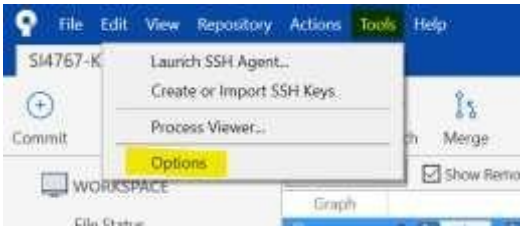




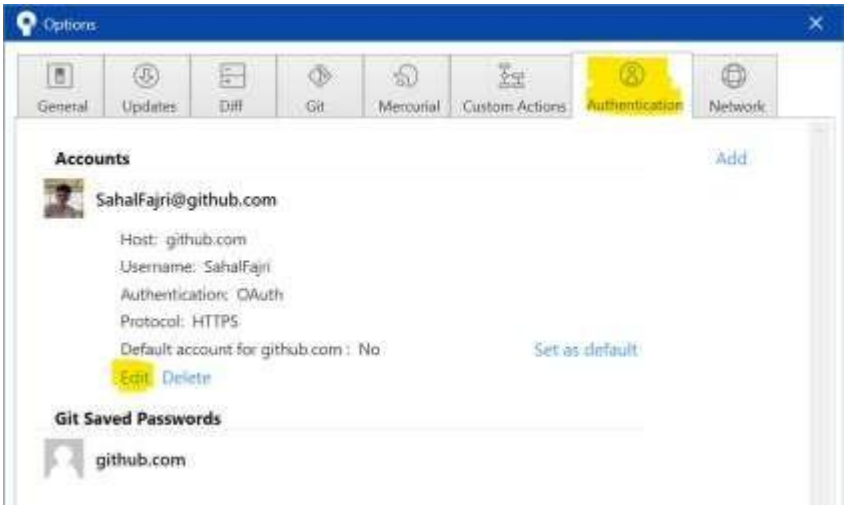
8. Klik **Generate token** di bagian bawah.
9. **Salin dan simpan token tersebut** karena tidak akan bisa melihatnya lagi setelah menutup halaman tersebut.



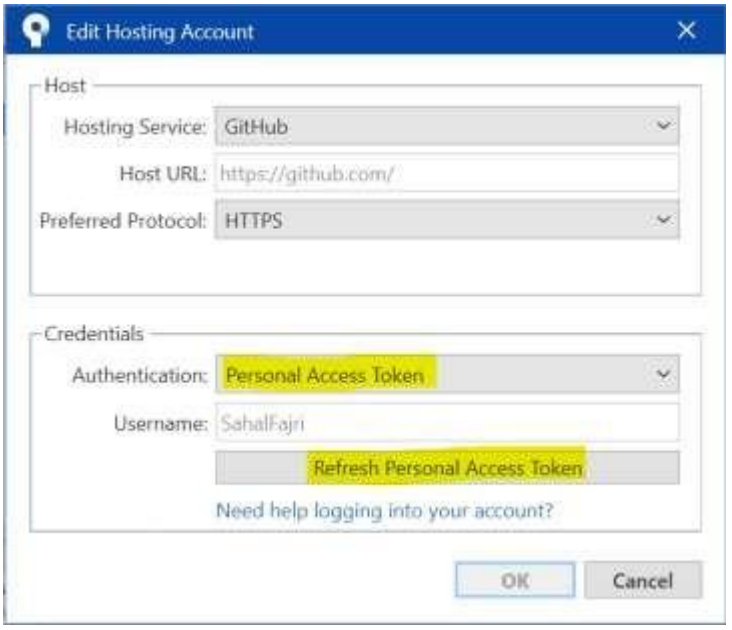
10. Buka **SourceTree**.
11. Pergi ke menu **Tools** > **Options**.



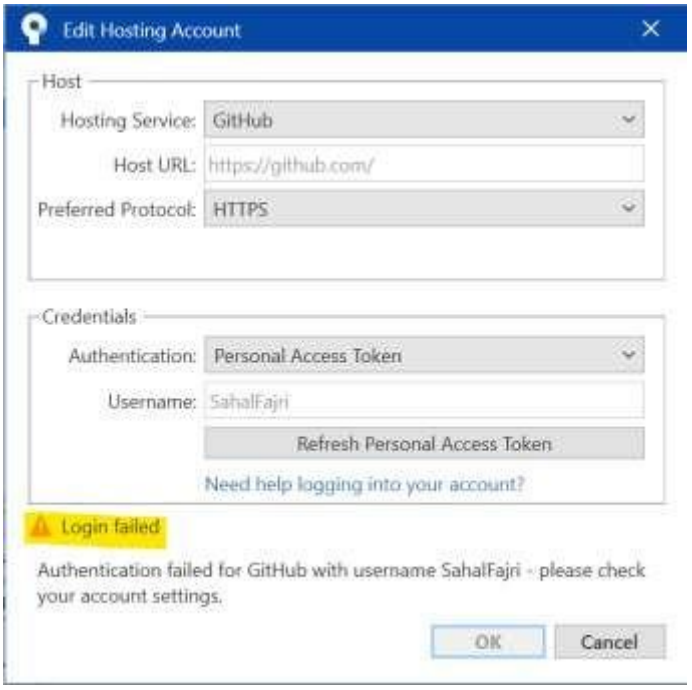
12. Pilih tab **Authentication**.
13. Cari akun GitHub kalian di daftar, lalu klik **Edit**.



14. Pada bagian **Auth Type**, pastikan terpilih **Personal Access Token**.
15. Klik **Refresh Personal Access Token** lalu langsung masukkan/tempel (paste) token baru yang kalian salin tadi ke kolom password/token.



16. Klik **OK**.
17. Jika muncul error **Login Failed** seperti pada gambar di bawah coba ikuti step berikut



Untuk Pengguna Windows:

1. Buka **Credential Manager** (cari di Start Menu).
2. Pilih **Windows Credentials**.
3. Cari entri yang berkaitan dengan **github.com** (termasuk SourceTree).
4. Klik entri tersebut lalu pilih **Remove**.
5. Coba lakukan **Push** lagi di SourceTree. SourceTree akan memunculkan pop-up login; masukkan username GitHub kalian dan **tempel PAT baru** kalian sebagai password-nya.

Untuk Pengguna macOS:

1. Buka **Keychain Access**.
2. Cari github.com.
3. Hapus entri tersebut.
4. Coba **Push** kembali di SourceTree dan masukkan PAT baru saat diminta.

"Do or do not.
There is no try"
- Grandmaster Yoda